

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

Национальный исследовательский ядерный университет «МИФИ»

Институт Интеллектуальных Кибернетических Систем

Отчёт

по курсовой работе на тему:

Разработка конечного автомата для обмена с внешней памятью SDRAM и
выполнения присвоений выходных сигналов в зависимости от текущего
состояния конечного автомата

Дисциплина: Схемотехника цифровых устройств

Подготовили студенты группы С23-501

Горбатенко И.А.

Серебрякова Д.Р.

Москва, 2026

Содержание

Содержание.....	2
Введение.....	3
Взаимосвязь с внешними модулями.....	4
Назначение и характеристики сигналов разработанного устройства.....	5
Внутренняя логика устройства.....	7
Состояния конечного автомата.....	8
Сдвиговой регистр для записи в SDRAM.....	11
Сдвиговой регистр для чтения из SDRAM.....	18
Тестирование.....	23
Синтез и компиляция схемы.....	30
Результаты прошивки ПЛИС.....	31
Заключение.....	36
Приложение.....	37

Введение

В рамках курсового проекта была разработана подсистема контроля обмена с внешней памятью в зависимости от состояний конечного автомата. Упрощённая структурная схема модели представлена на Рисунке 1.

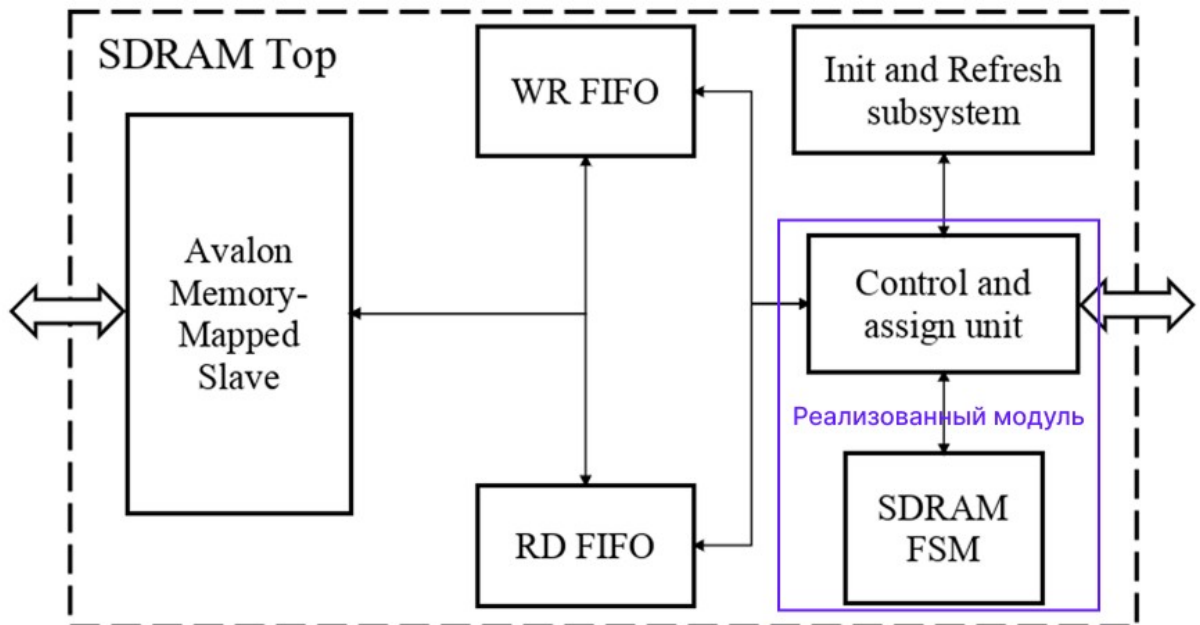


Рисунок 1 — Упрощённая структурная схема контроллера SDRAM

Требования к проекту:

- 1) Ширина шины данных, поступающая из интерфейса Avalon Memory-Mapped, фиксирована и составляет 64 бита;
- 2) Отдельная буферизация для чтения и записи;
- 3) Параметры SDRAM должны задаваться через generics: burst = {1, 2, 4, 8}, word_width = {8, 16, 32, 64} - ширина данных для внешней памяти;
- 4) Режим адресации для SDRAM – последовательный (Sequential Mode).

Тестирование проводилось на ПЛИС 10CL025YU256C8G и SDRAM W9864G6JT-6.

В данной части работы был разработан конечный автомат для обмена с внешней памятью SDRAM и выполнения присвоений выходных сигналов в зависимости от текущего состояния конечного автомата.

Взаимосвязь с внешними модулями

Модуль Control unit должен корректно взаимодействовать с модулем Init and Refresh subsystem, SDRAM и с FIFO.

Для реализации блоков Init and Refresh subsystem и Control unit в структуре контроллера SDRAM используется мультиплексор Assign unit, который позволяет формировать сигналы, поступающие в SDRAM, только из одного модуля (Init and Refresh subsystem или Control unit). Схема соединения конечного автомата, подсистемы инициализации, обновления и предварительной зарядки, мультиплексора, памяти и буфера FIFO представлена на Рисунке 2.

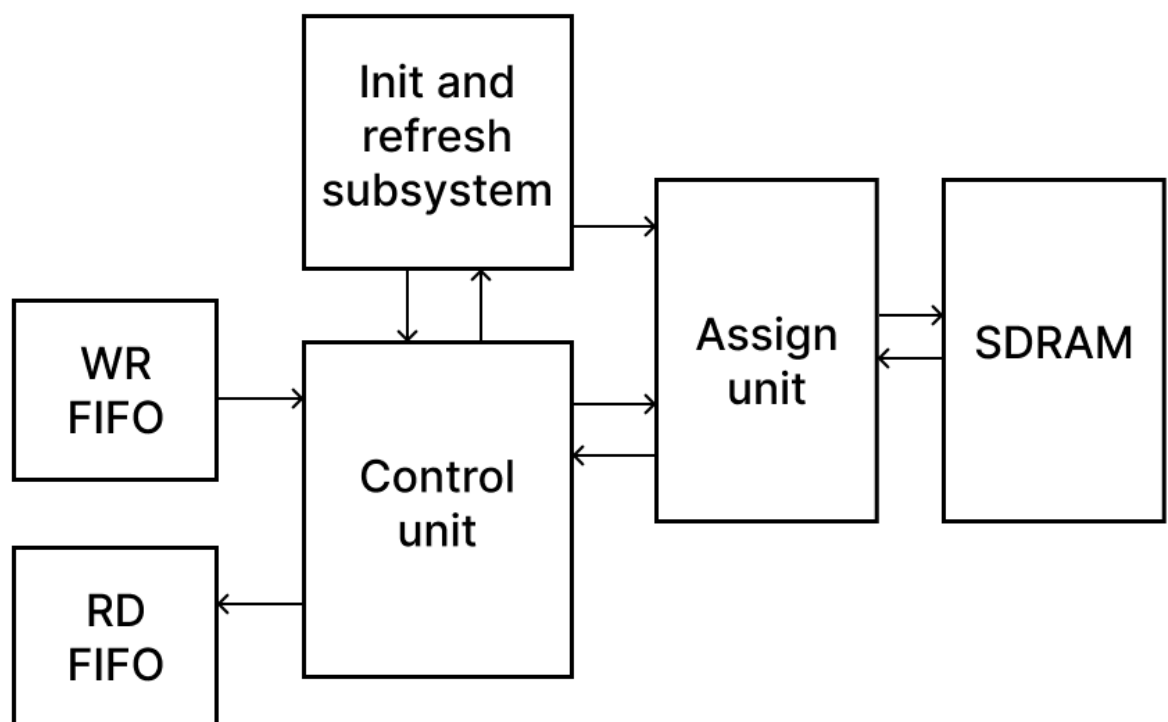


Рисунок 2 — Схема соединения контроллера и внешних модулей

Подсистема контроля обмена с внешней памятью в зависимости от состояний конечного автомата выполняет следующие задачи:

- Запись данных в память из FIFO в соответствии с поступающей командой;
- Чтение данных из памяти и формирование шины для FIFO;
- Корректная передача управления шиной SDRAM через Assign unit и завершение текущей транзакции перед передачей управления Init and Refresh subsystem.

Назначение и характеристики сигналов разработанного устройства

Условное графическое изображение разработанного устройства приведено на Рисунке 3.

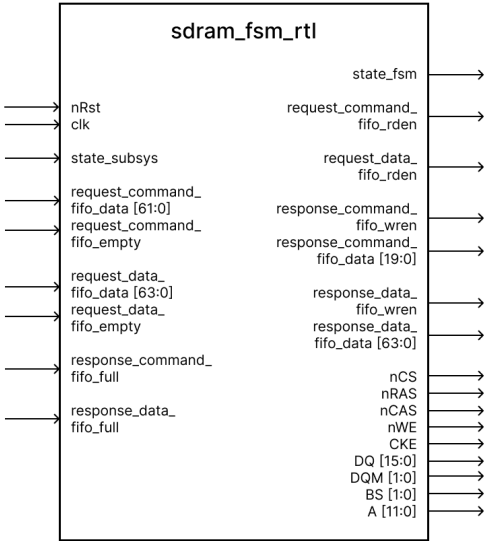


Рисунок 3 — УГО FSM

В Таблице 1 приведено описание входных и выходных сигналов разработанного конечного автомата.

Таблица 1. Входные и выходные сигналы конечного автомата

Название сигнала	Вход/выход	Разрядность, бит	Описание
Clk	in	1	Тактовый сигнал
nRst	in	1	Сигнал асинхронного сброса
state_subsys	in	-	Сигнал, определяющий состояние подсистемы Init and Refresh subsystem
state_fsm	out	-	Сигнал, определяющий состояние подсистемы FSM
request_command_fifo_rden	out	1	Сигнал активации чтения команды из FIFO
request_command_fifo_data	in	62	Входная 62-битная шина команды из FIFO

request_command_fifo_empty	in	1	Сигнал пустоты команды из FIFO
request_data_fifo_rden	out	1	Сигнал активации чтения данных из FIFO
request_data_fifo_data	in	64	Входная 64-битная шина данных из FIFO
request_data_fifo_empty	in	1	Сигнал пустоты данных из FIFO
response_command_fifo_wren	out	1	Сигнал активации записи в FIFO
response_command_fifo_data	out	20	Выходная 20-битная шина команды для FIFO
response_command_fifo_full	in	1	Сигнал заполненности команды из FIFO
response_data_fifo_wren	out	1	Сигнал активации записи данных в FIFO
response_data_fifo_data	out	64	Выходная 64-битная шина данных для FIFO
response_data_fifo_full	in	1	Сигнал заполненности данных в FIFO
nCS	out	1	Сигнал выбора микросхемы памяти. «1» - игнорирование содержимого на шине, «0» - просмотр команд
nRAS	out	1	Сигнал для строки в памяти
nCAS	out	1	Сигнал для столбца в памяти
nWE	out	1	Сигнал определения записи. «0» - операция записи, «1» - в противном случае
CKE	out	1	Сигнал реакции на тактовый сигнал
DQ	out	16	16-битная шина данных для записи в память
DQM	out	2	Маска для записи данных в память
BS	out	2	Вектор для выбора банка памяти
A	out	12	Адресная 12-битная шина

--	--	--	--

Внутренняя логика устройства

Контроллер состоит из FSM, определяющего состояние, и двух сдвиговых регистров для записи и чтения данных. На Рисунке 4 показана структурная схема внутренней логики разработанного устройства.

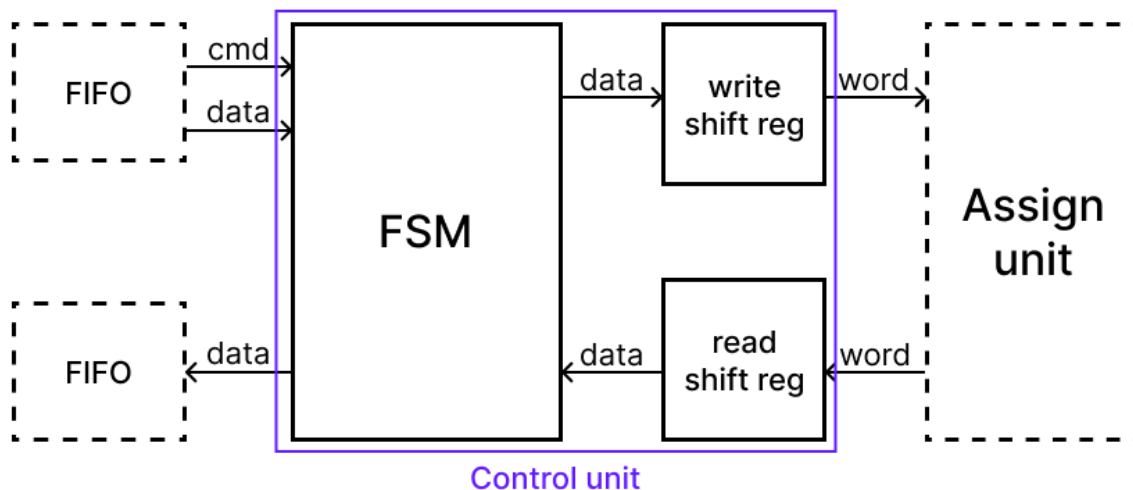


Рисунок 4. Структурная схема внутренней логики

Из FIFO поступает команда и данные для записи в SDRAM, которые попадают в контроллер и в состоянии записи подаются в сдвиговый регистр для записи, где формируются слова и маска для них. При чтении данные из памяти поступают в сдвиговый регистр для чтения, где формируется выходная 64-битная шина и подаётся на вход FIFO.

Состояния конечного автомата

Диаграмма состояний разработанного конечного автомата представлена на Рисунках 5-6 в виде графа состояний.

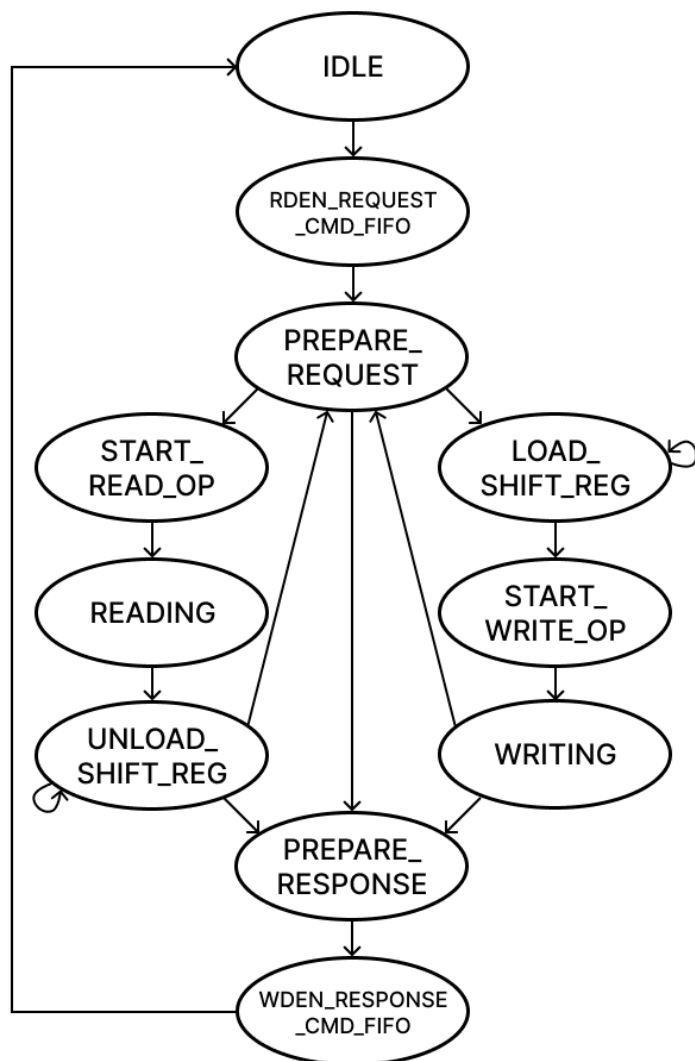


Рисунок 5. Диаграмма состояний конечного автомата для FIFO

Состояния конечного автомата для FIFO:

- IDLE – состояние ожидания команды из FIFO;
- RDEN_REQUEST_CMD_FIFO – состояние чтения команды из FIFO;
- PREPARE_REQUEST – состояние подготовки данных для очередной транзакции к памяти;
- START_READ_OP – состояние начала операции чтения из FIFO;
- READING – состояние выполнения операции чтения из FIFO;
- UNLOAD_SHIFT_REG – состояние выгрузки регистра для записи в память;
- LOAD_SHIFT_REG – состояние загрузки регистра для чтения из памяти;

- START_WRITE_OP – состояние начала операции записи в FIFO;
- WRITING – состояние выполнения операции записи в FIFO;
- PREPARE_RESPONSE – состояние подготовки ответа о выполненной операции;
- WREN_RESPONSE_CMD_FIFO – состояние записи ответа в FIFO о выполненной операции.

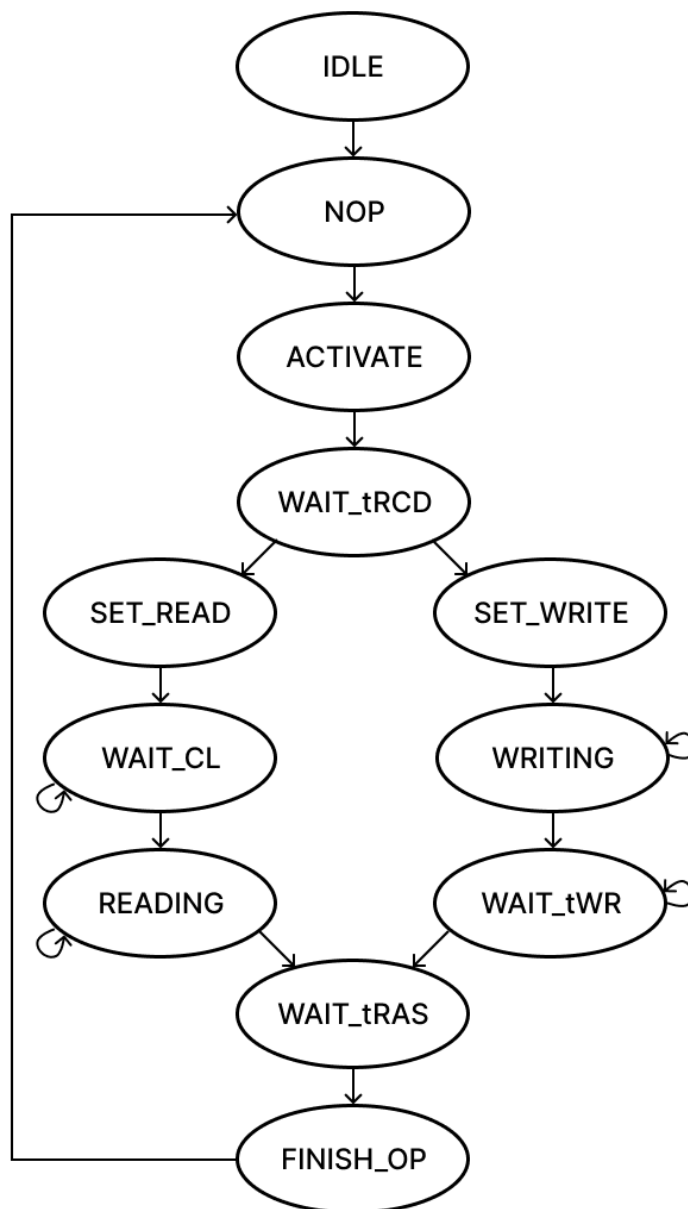


Рисунок 6. Диаграмма состояний конечного автомата для SDRAM

Состояния конечного автомата для SDRAM:

- IDLE – состояние инициализации;
- NOP – состояние ожидания команды для SDRAM;
- ACTIVATE – состояние открытия банка и строки в памяти;

- WAIT_tRCD – состояние временной задержки для открытия банка и строки в памяти;
- SET_READ – состояние установки операции чтения из памяти;
- WAIT_CL – состояние временной задержки для установки операции чтения из памяти;
- READING – состояние чтения из памяти;
- SET_WRITE – состояние установки операции записи в память;
- WRITING – состояние записи в память;
- WAIT_tWR – состояние временной задержки после записи в память;
- WAIT_tRAS – состояние временной задержки, которая длится от состояния ACTIVATE до подачи PRECHARGE;
- FINISH_OP – состояние завершения операции, происходящей с памятью.

Сдвиговый регистр для записи в SDRAM

Для операции записи из FIFO был реализован сдвиговый регистр, УГО которого приведено на Рисунке 7.

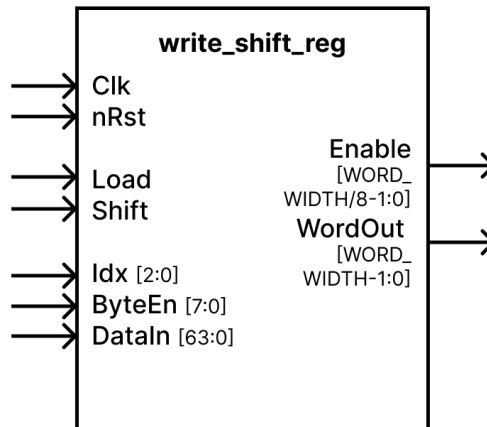


Рисунок 7. УГО сдвигового регистра для записи

В Таблице 2 приведено описание входных и выходных сигналов разработанного регистра.

Таблица 2. Входные и выходные сигналы регистра для записи

Название сигнала	Вход/выход	Разрядность, бит	Описание
Clk	in	1	Тактовый сигнал
nRst	in	1	Сигнал асинхронного сброса
Load	in	1	Сигнал разрешения загрузки. «1» - если регистр пуст и его можно заполнить, «0» - в противном случае
Shift	in	1	Сигнал сдвига данных. «1» - если на выходе нужно получить сдвиг, «0» - в противном случае
Idx	in	3	Номер, по которому производится расчёт стартового индекса данных для занесения в регистр
ByteEn	in	8	Байт разрешения записи 8-битных данных, соответствующих биту из него
DataIn	in	64	Шина входных данных для сдвига
Enable	out	world_width/8	Сигнал разрешения записи в память выходной

			порции данных
WordOut	out	world_width	Выходная порция данных

Принцип работы регистра заключается в параллельном занесении $\text{burst} \times \text{world_width}$ бит из входной 64-битной шины данных в него и последовательном сдвиге в количестве world_width бит за один такт. При этом с помощью параметров системы задаются значения burst и world_width . Все возможные их комбинации приведены в Таблице 3 и рассмотрены для работы регистра.

Таблица 3. Комбинации значений burst и world_width и соответствующие им параметры

burst	world_width	Максимальное количество битов в регистре (FRAG_BITS)	Количество записей в регистр из 64-битной шины
1	8	8	8
1	16	16	4
1	32	32	2
1	64	64	1
2	8	16	4
2	16	32	2
2	32	64	1
2	64	128	1
4	8	32	2
4	16	64	1
4	32	128	1
4	64	256	1
8	8	64	1
8	16	128	1
8	32	256	1
8	64	512	1

На Рисунке 8 приведена схема работы сдвигового регистра и его взаимодействие с входными и выходными данными в случае $\text{burst} = 4$ и $\text{world_width} = 16$.

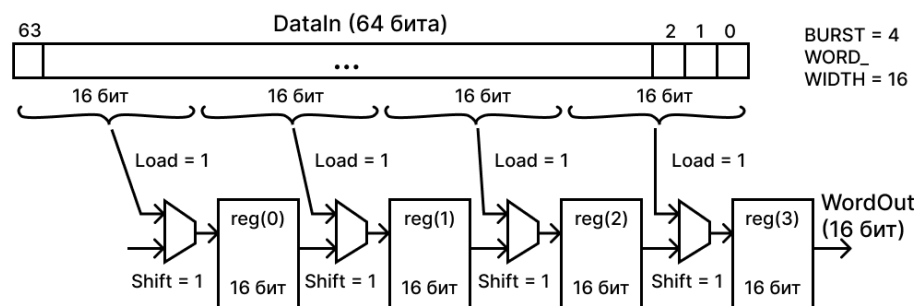


Рисунок 8. Схема работы сдвигового регистра для записи при burst = 4 и world_width = 16

На Рисунке 9 приведена схема работы сдвигового регистра и его взаимодействие с входными и выходными данными в случае burst = 2 и world_width = 8.

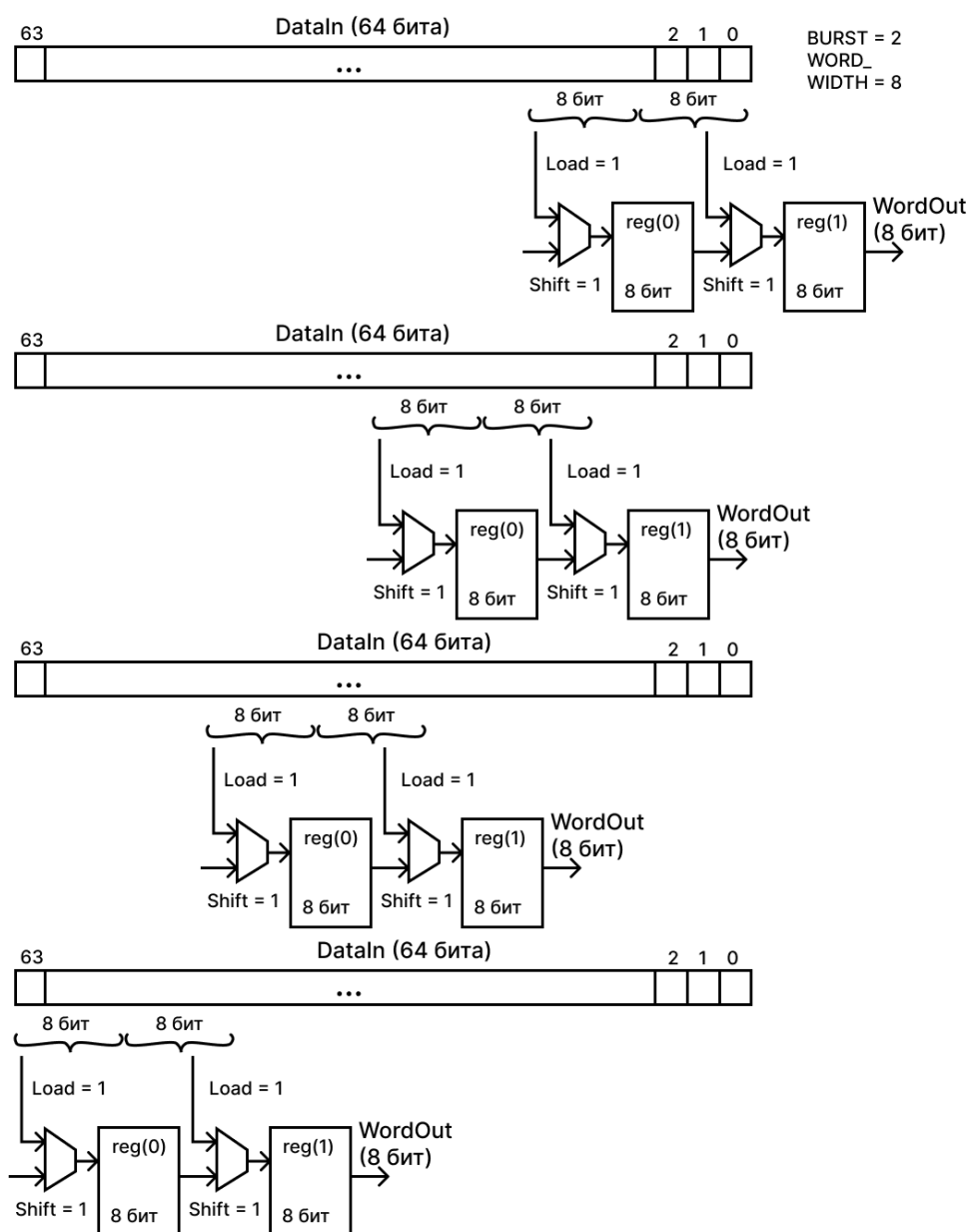


Рисунок 9. Схема работы сдвигового регистра для записи при burst = 2 и world_width = 8

На Рисунке 10 приведена схема работы сдвигового регистра и его взаимодействие с входными и выходными данными в случае burst = 8 и world_width = 32.

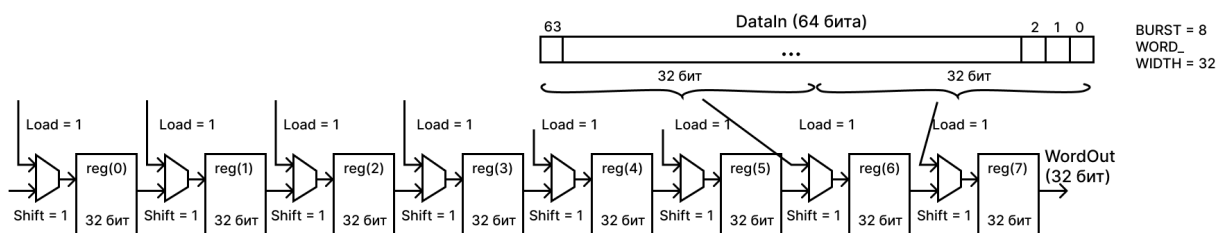


Рисунок 10. Схема работы сдвигового регистра для записи при burst = 8 и world_width = 32

На вход регистра подаётся 64-битная шина данных, которая загружается в регистр за один такт путём подачи единицы на вход Load. После этого происходит сдвиг порции по world_width бит при выставлении сигнала Shift единицей и выдача этих выходных данных за один такт. Одновременная подача единицы на Load и Shift невозможна, потому что Shift будет в приоритете из условия, приведённого в коде разработанного регистра. Возможны несколько ситуаций для подачи загрузки и сдвига, которые рассматриваются в работе регистра:

- Если общее количество битов в регистре равно 64, то есть FRAG_BITS = 64, то индекс Idx подаётся один раз в значении «000», происходит одна загрузка, после чего burst сдвигов;
- Если общее количество битов в регистре меньше 64, то есть FRAG_BITS < 64, то происходит загрузка и burst сдвигов в количестве 64/FRAG_BITS раз, при этом каждую загрузку инкрементируется индекс Idx, начиная со значения «000» (в случае burst = 1 и world_width = 8 последняя загрузка регистра будет происходить при Idx = «111»);
- Если общее количество битов в регистре больше 64, то есть FRAG_BITS > 64, то индекс Idx подаётся один раз в значении «000», происходит одна загрузка, после чего burst сдвигов, а оставшиеся свободные биты в регистре заполняются нулями.

На вход регистра подаётся байт разрешения ByteEn для записи в память, каждый бит которого соответствует одному байту из входной 64-битной шины. ByteEn может принимать значения «11111111» (все входные биты разрешены для записи в память), «01111111» (разрешены для записи в память последние 7 входных байтов), «00111111» (разрешены для записи в память последние 6 входных байтов), ..., «00000001» (разрешены

для записи в память последний 1 входной байт), «00000000» (ни один входной бит не разрешён для записи в память), «10000000» (разрешены для записи в память первый 1 входной байт), ..., «11111110» (разрешены для записи в память первые 7 входных байтов), а также комбинации, которые внутри содержат единицы, а по краям нули, например, ByteEn = «01111100». Для случая world_width = 8 каждый выходной бит разрешения соответствует биту из входного байта, а при других значениях world_width выходной Enable имеет размер world_width/8 и соответствует разрешению записи выходных битов.

RTL-схема сдвигового регистра со значениями burst = 4 и world_width = 16 для записи представлена на Рисунке 11.

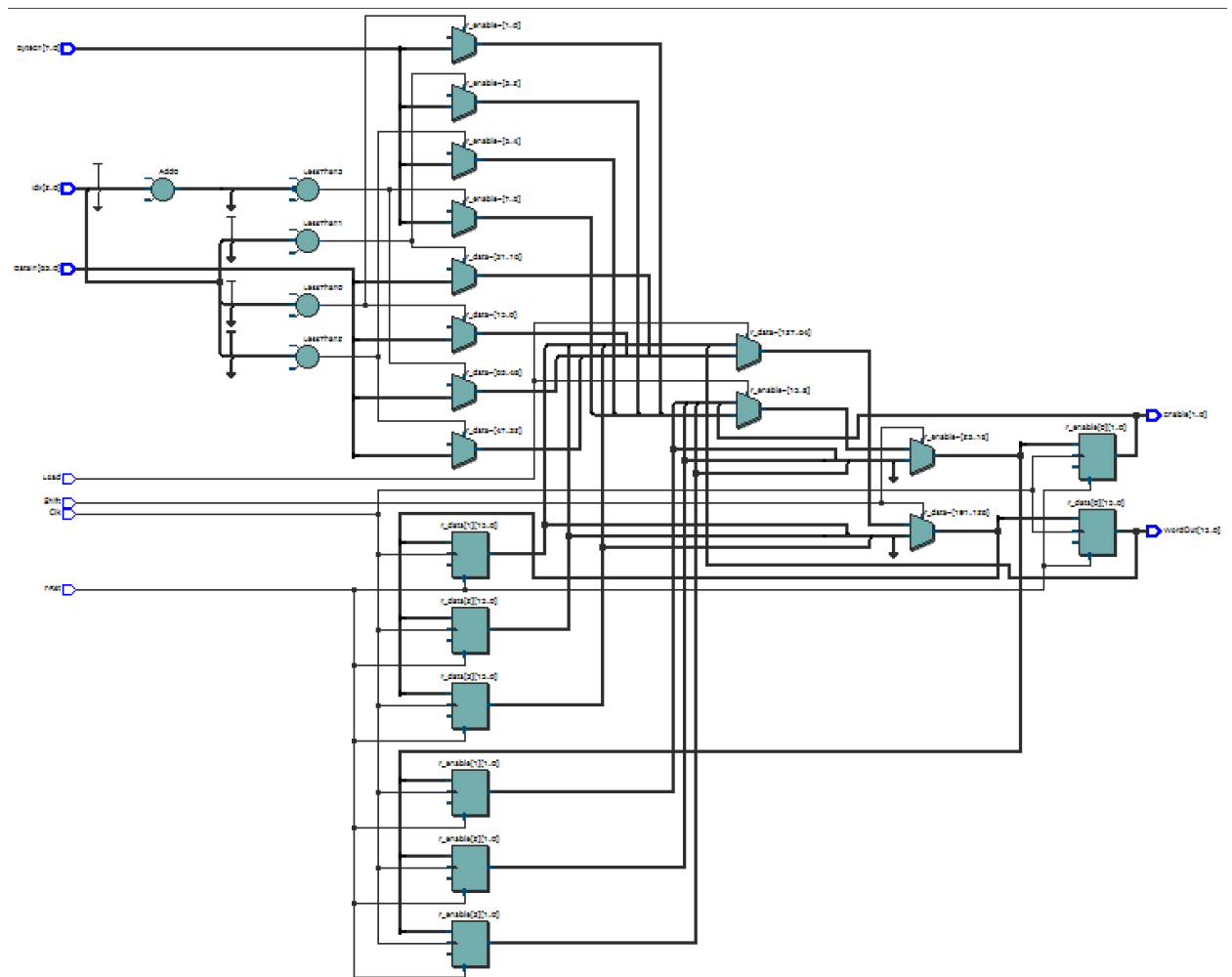


Рисунок 11. RTL-схема сдвигового регистра для записи со значениями burst = 4 и world_width = 16

Исходный код разработанного сдвигового регистра для записи из FIFO приведён в Приложении 1.

Для регистра были написаны тесты, демонстрирующие его работу при всех возможных комбинациях значений burst и world_width, а также при различных значениях ByteEn. Тестирование проводилось по данным, представленным в Таблице 4, результаты показаны на Рисунках 12-14. Исходный код тестбэнча и тестера приведены в Приложениях 2-3.

Таблица 4. Проведённые тесты для регистра для записи

№ теста	Описание	Ожидаемый результат	Статус
1	burst = 4, world_width = 16, ByteEn = «11111000» 1 загрузка, 4 сдвига	Старшие 5 байт на выходе будут валидны	Прошёл
2	burst = 2, world_width = 8, ByteEn = «11111111» 4 загрузки, 8 сдвигов	Все байты на выходе будут валидны	Прошёл
3	burst = 8, world_width = 32, ByteEn = «01111100» 1 загрузка, 2 сдвига	Со второго по шестой байт на выходе будут валидны	Прошёл

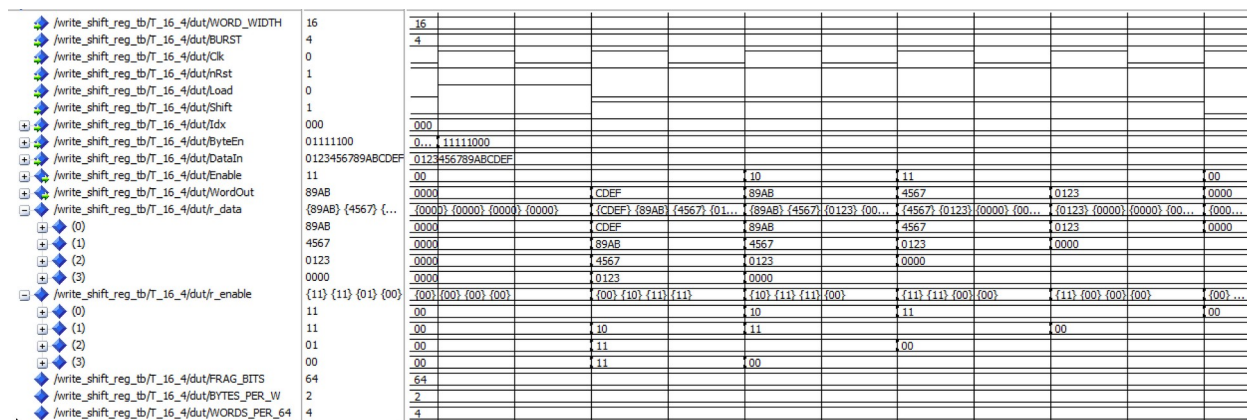


Рисунок 12. Фрагмент временной диаграммы выполнения операции сдвига в регистре для записи для теста №1

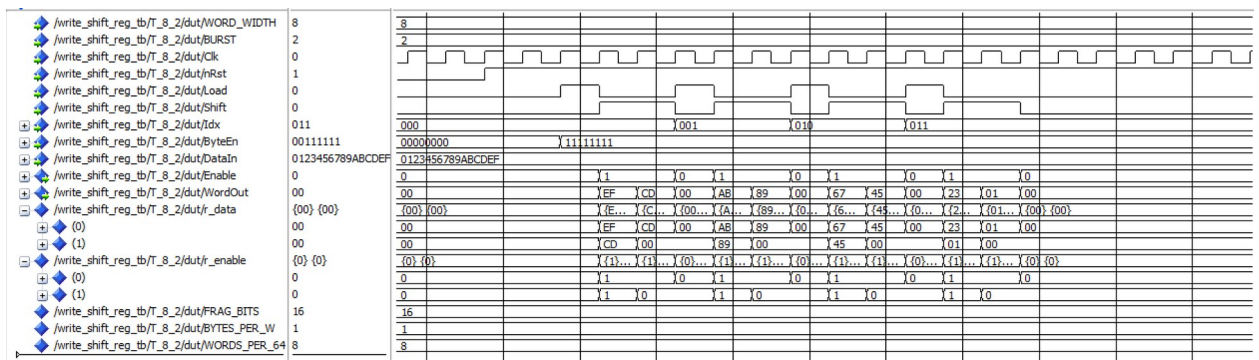


Рисунок 13. Фрагмент временной диаграммы выполнения операции сдвига в регистре для записи для теста №2

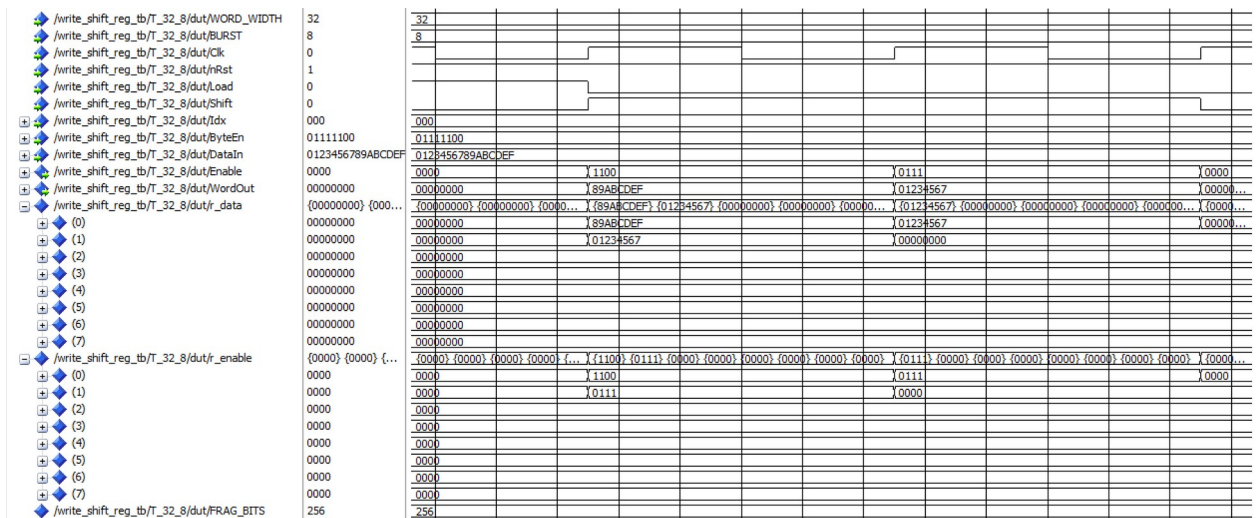


Рисунок 14. Фрагмент временной диаграммы выполнения операции сдвига в регистре для записи для теста №3

Сдвиговый регистр для чтения из SDRAM

Для операции чтения из SDRAM был реализован сдвиговый регистр, УГО которого приведено на Рисунке 15.

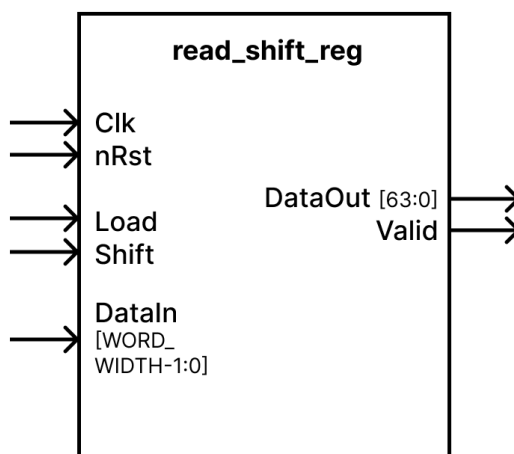


Рисунок 15. УГО сдвигового регистра для чтения

В Таблице 5 приведено описание входных и выходных сигналов разработанного регистра.

Таблица 5. Входные и выходные сигналы регистра для чтения

Название сигнала	Вход/выход	Разрядность, бит	Описание
Clk	in	1	Тактовый сигнал
nRst	in	1	Сигнал асинхронного сброса
Load	in	1	Сигнал разрешения загрузки. «1» - если регистр пуст и его можно заполнить, «0» - в противном случае
Shift	in	1	Сигнал сдвига данных. «1» - если на выходе нужно получить сдвиг, «0» - в противном случае
DataIn	in	world_width	Шина входных данных из памяти для записи в регистр
DataOut	out	64	Выходная порция данных, записываемая в FIFO
Valid	out	1	Сигнал валидности выходных данных. «1» - если данные валидны, «0» - в противном случае

На Рисунке 16 приведена схема работы сдвигового регистра и его взаимодействие с входными и выходными данными в случае $\text{burst} = 4$ и $\text{world_width} = 16$.

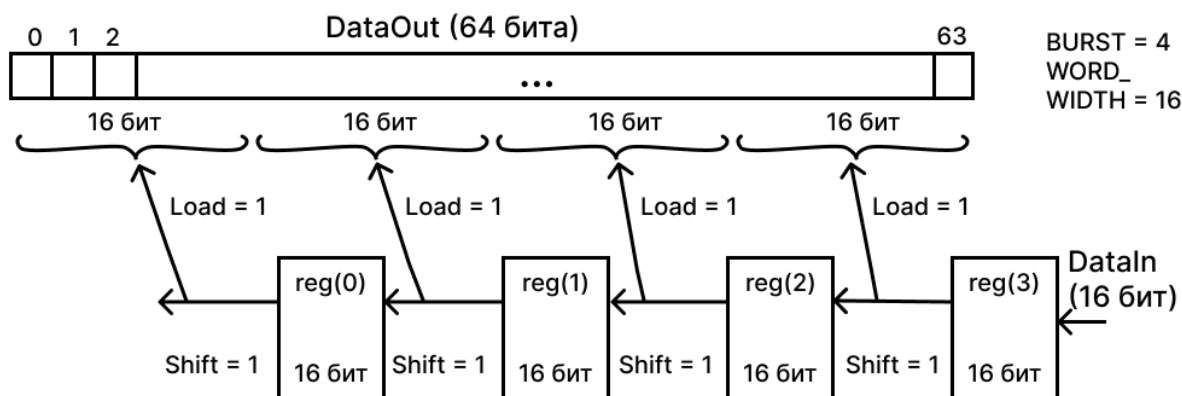


Рисунок 16. Схема работы сдвигового регистра для чтения при $\text{burst} = 4$ и $\text{world_width} = 16$

На Рисунке 17 приведена схема работы сдвигового регистра и его взаимодействие с входными и выходными данными в случае $\text{burst} = 2$ и $\text{world_width} = 8$.

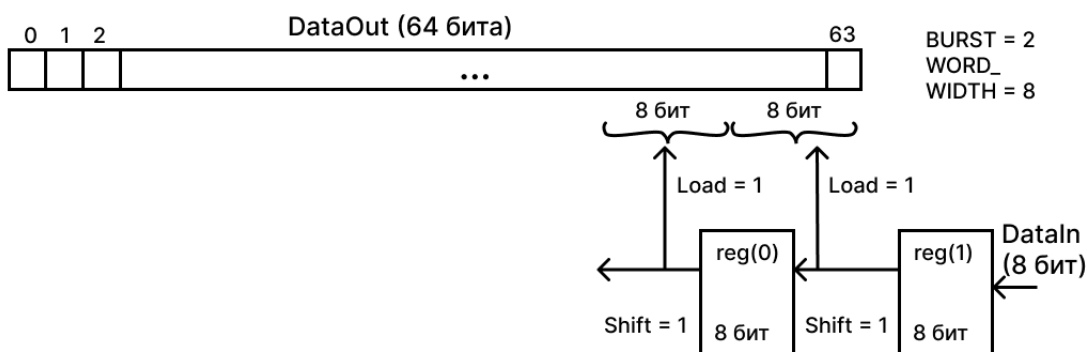


Рисунок 17. Схема работы сдвигового регистра для записи при $\text{burst} = 2$ и $\text{world_width} = 8$

На Рисунке 18 приведена схема работы сдвигового регистра и его взаимодействие с входными и выходными данными в случае $\text{burst} = 8$ и $\text{world_width} = 32$.

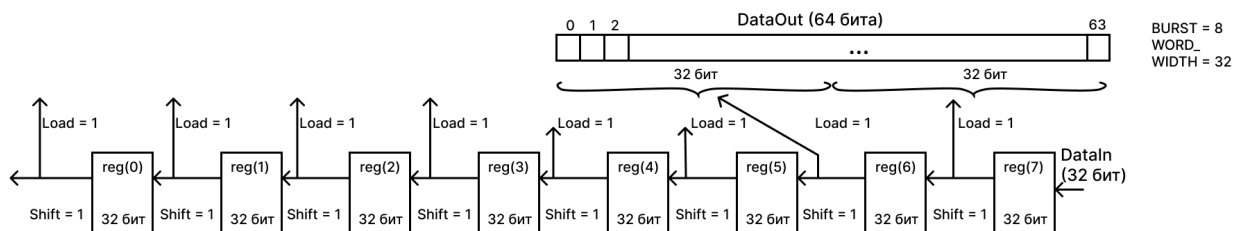


Рисунок 18. Схема работы сдвигового регистра для записи при $\text{burst} = 8$ и $\text{world_width} = 32$

Принцип работы регистра заключается в параллельном занесении $\text{burst} * \text{world_width}$ бит из памяти в него и последовательном сдвиге данных в количестве world_width бит за один такт. При этом с помощью параметров системы задаются значения burst и world_width . Все возможные их комбинации приведены в Таблице 3 и рассмотрены для работы регистра.

На вход регистра подаётся world_width бит данных, которые загружаются в регистр за один такт путём подачи единицы на вход Shift. После этого происходит сдвиг порции по world_width бит и занесение в регистр новой порции данных из памяти при выставлении сигнала Shift единицей. После заполнения регистра входными данными подаётся единица на вход Load, за один такт данные из регистра выгружаются в выходную 64-битную шину и сигнал Valid становится единицей. Одновременная подача единицы на Load и Shift невозможна, потому что Shift будет в приоритете из условия, приведённого в коде разработанного регистра. Возможны несколько ситуаций для подачи загрузки и сдвига, которые рассматриваются в работе регистра:

- Если общее количество битов в регистре равно 64, то есть $\text{FRAG_BITS} = 64$, то происходит burst сдвигов, после чего одна загрузка;
- Если общее количество битов в регистре меньше 64, то есть $\text{FRAG_BITS} < 64$, то происходит burst сдвигов и загрузка выходных данных, которые будут дозаполнены нулями в старших битах;
- Если общее количество битов в регистре больше 64, то есть $\text{FRAG_BITS} > 64$, то происходит burst сдвигов, после чего загрузка выходной полностью заполненной 64-битной шины, такие итерации повторяются $\text{FRAG_BITS}/64$ раз.

RTL-схема сдвигового регистра со значениями $\text{burst} = 4$ и $\text{world_width} = 16$ для записи представлена на Рисунке 19.

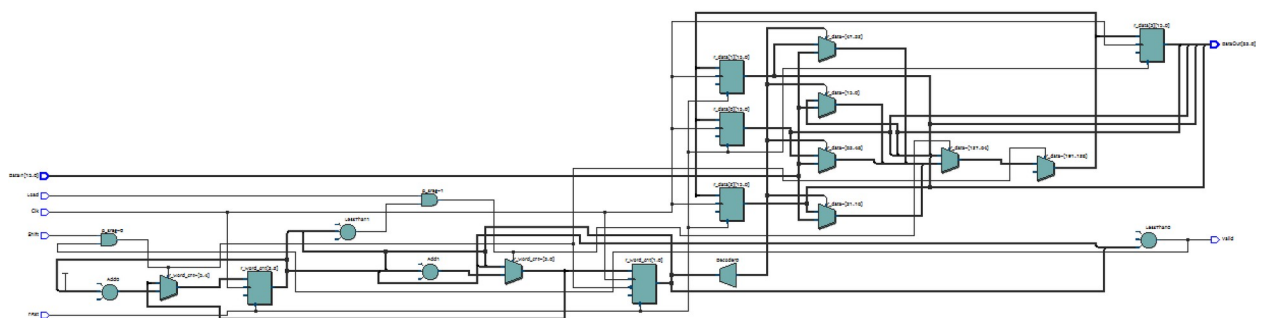


Рисунок 19. RTL-схема сдвигового регистра для чтения со значениями $\text{burst} = 4$ и $\text{world_width} = 16$

Исходный код разработанного сдвигового регистра для чтения из SDRAM приведён в Приложении 4.

Для регистра были написаны тесты, демонстрирующие его работу при всех возможных комбинациях значений burst и world_width. Тестирование проводилось по данным, представленным в Таблице 6, результаты показаны на Рисунках 20-22. Исходный код тестбэнча и тестера приведены в Приложениях 5-6.

Таблица 6. Проведённые тесты для регистра для чтения

№ теста	Описание	Ожидаемый результат	Статус
1	burst = 4, world_width = 16 1 загрузка, 4 сдвига	Входные 8 байт записываются в одну выходную 64-битную шину	Прошёл
2	burst = 2, world_width = 8 1 загрузка, 2 сдвига	Входные 2 байта записываются в регистр, на выходе шина содержит 2 байта данных и нулевые элементы	Прошёл
3	burst = 8, world_width = 32 4 загрузки, 8 сдвигов	Входные слова будут записаны в регистр, затем выгружены в выходную шину последовательно 4 раза	Прошёл

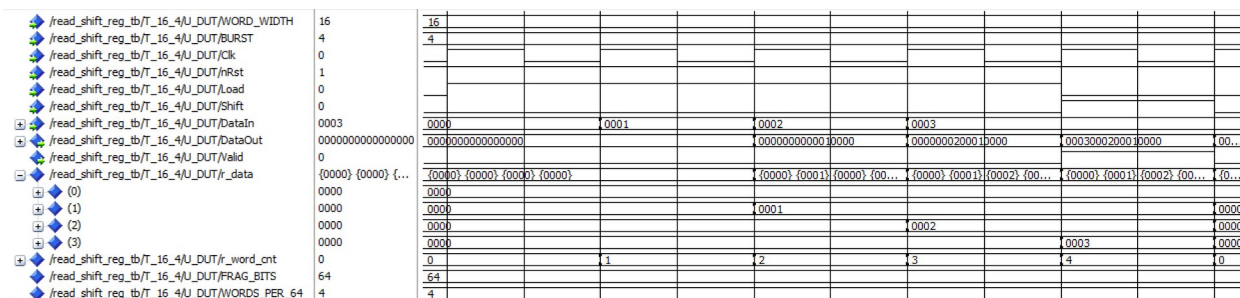


Рисунок 20. Фрагмент временной диаграммы выполнения операции сдвига в регистре для чтения для теста №1

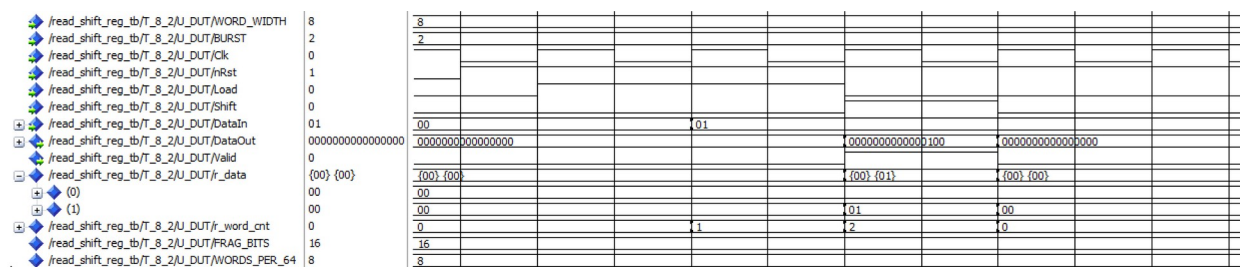


Рисунок 21. Фрагмент временной диаграммы выполнения операции сдвига в регистре для чтения для теста №2

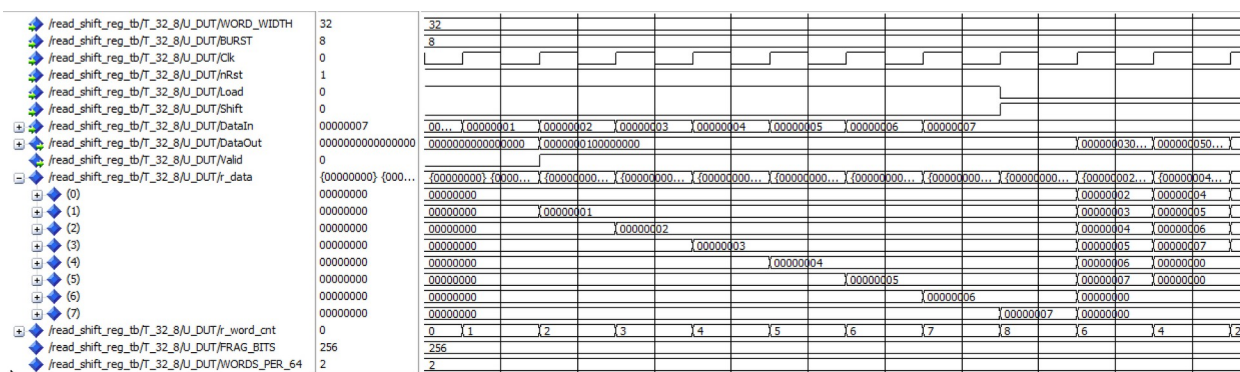


Рисунок 22. Фрагмент временной диаграммы выполнения операции сдвига в регистре для чтения для теста №3

Тестирование

Для верификации подсистемы были разработаны тесты, перечисленные в Таблице 7 и показанные на Рисунках 23-32.

Таблица 7. Проведённые тесты для конечного автомата

№ теста	Описание	Ожидаемый результат	Статус
1	Начальная инициализация и смена состояний	После начального состояния IDLE происходит переход в состояние RDEN_REQUEST_CMD_FIFO при подаче «1» на request_command_rden_r. Спустя такт происходит переключение в состояние PREPARE_REQUEST	Прошёл
2	Вычисление адреса в памяти	Для каждой порции данных, записываемых в память, происходит расчёт адреса по строке и столбцу, который увеличивается по значению столбца, а при переносе – по строке. При чтении данных обращение происходит по тому же адресу, по которому были записаны данные	Прошёл
3	Определение операции чтения	При подаче «1» на op_type_r начинается операция чтения. Она различается сменой состояний START_READ_OP -> READING	Прошёл
4	Определение операции записи	При подаче «0» на op_type_r начинается операция записи. Она различается сменой состояний SET_WRITE -> WRITING	Прошёл
5	Операция пакетной записи	В состоянии WRITING происходит корректная запись порций данных burst = 8 раз в память и соблюдены временные параметры для подачи	Прошёл

		данных	
6	Операция пакетного чтения	В состоянии READING происходит корректное чтение порции данных burst = 8 раз из памяти и соблюдены временные параметры для подачи данных	Прошёл
7	Операция одиночной записи	В состоянии WRITING происходит корректная запись одной порций данных в память и соблюдены временные параметры для подачи данных	Прошёл
8	Операция одиночного чтения	В состоянии READING происходит корректное чтение одной порции данных из памяти и соблюдены временные параметры для подачи данных	Прошёл
9	Изменение строки и банка в памяти	После записи полной строки происходит инкрементация её номера и запись данных с новой строки. При заполнении банка происходит запись в новый банк с нулевой строки и столбца	Прошёл
10	Взаимодействие с модулем Init and Refresh subsystem	Когда в модуле Init and Refresh subsystem не установлено состояние ValidOp, контроллер должен находиться в состоянии Waiting. В противном случае можно переходить в другие возможные состояния	Прошёл

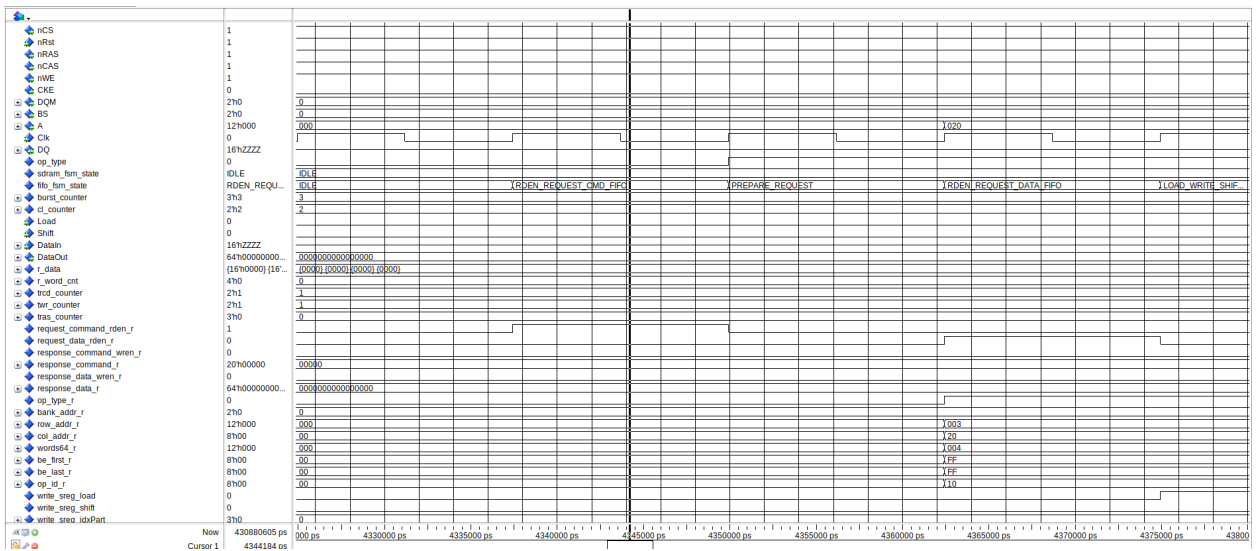


Рисунок 23. Фрагмент временной диаграммы для теста №1

```
# sdramtoptb.U_SDRAM : at time 404794.0 ns WRITE: Bank = 0 Row = 3, Col = 32, Data = 1
# sdramtoptb.U_SDRAM : at time 404806.0 ns WRITE: Bank = 0 Row = 3, Col = 33, Data = 0
# sdramtoptb.U_SDRAM : at time 404819.0 ns WRITE: Bank = 0 Row = 3, Col = 34, Data = 0
# sdramtoptb.U_SDRAM : at time 404831.0 ns WRITE: Bank = 0 Row = 3, Col = 35, Data = 0
# sdramtoptb.U_SDRAM : at time 405006.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 405044.0 ns WRITE: Bank = 0 Row = 3, Col = 36, Data = 2
# sdramtoptb.U_SDRAM : at time 405056.0 ns WRITE: Bank = 0 Row = 3, Col = 37, Data = 0
# sdramtoptb.U_SDRAM : at time 405069.0 ns WRITE: Bank = 0 Row = 3, Col = 38, Data = 0
# sdramtoptb.U_SDRAM : at time 405081.0 ns WRITE: Bank = 0 Row = 3, Col = 39, Data = 0
# sdramtoptb.U_SDRAM : at time 405256.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 405294.0 ns WRITE: Bank = 0 Row = 3, Col = 40, Data = 3
# sdramtoptb.U_SDRAM : at time 405306.0 ns WRITE: Bank = 0 Row = 3, Col = 41, Data = 0
# sdramtoptb.U_SDRAM : at time 405319.0 ns WRITE: Bank = 0 Row = 3, Col = 42, Data = 0
# sdramtoptb.U_SDRAM : at time 405331.0 ns WRITE: Bank = 0 Row = 3, Col = 43, Data = 0
# sdramtoptb.U_SDRAM : at time 405506.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 405544.0 ns WRITE: Bank = 0 Row = 3, Col = 44, Data = 4
# sdramtoptb.U_SDRAM : at time 405556.0 ns WRITE: Bank = 0 Row = 3, Col = 45, Data = 0
# sdramtoptb.U_SDRAM : at time 405569.0 ns WRITE: Bank = 0 Row = 3, Col = 46, Data = 0
# sdramtoptb.U_SDRAM : at time 405581.0 ns WRITE: Bank = 0 Row = 3, Col = 47, Data = 0
# ** Note: RESP_CMD: words64=4 op_id=16
# Time: 405781007 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 405794.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 405861.0 ns READ : Bank = 0 Row = 3, Col = 32, Data = 1
# sdramtoptb.U_SDRAM : at time 405874.0 ns READ : Bank = 0 Row = 3, Col = 33, Data = 0
# sdramtoptb.U_SDRAM : at time 405886.0 ns READ : Bank = 0 Row = 3, Col = 34, Data = 0
# sdramtoptb.U_SDRAM : at time 405899.0 ns READ : Bank = 0 Row = 3, Col = 35, Data = 0
# ** Note: RESP_DATA: 0x0000000000000001
# Time: 406043503 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406056.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 406124.0 ns READ : Bank = 0 Row = 3, Col = 36, Data = 2
# sdramtoptb.U_SDRAM : at time 406136.0 ns READ : Bank = 0 Row = 3, Col = 37, Data = 0
# sdramtoptb.U_SDRAM : at time 406149.0 ns READ : Bank = 0 Row = 3, Col = 38, Data = 0
# sdramtoptb.U_SDRAM : at time 406161.0 ns READ : Bank = 0 Row = 3, Col = 39, Data = 0
# ** Note: RESP_DATA: 0x0000000000000002
# Time: 406305999 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406318.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 406386.0 ns READ : Bank = 0 Row = 3, Col = 40, Data = 3
# sdramtoptb.U_SDRAM : at time 406399.0 ns READ : Bank = 0 Row = 3, Col = 41, Data = 0
# sdramtoptb.U_SDRAM : at time 406411.0 ns READ : Bank = 0 Row = 3, Col = 42, Data = 0
# sdramtoptb.U_SDRAM : at time 406424.0 ns READ : Bank = 0 Row = 3, Col = 43, Data = 0
# ** Note: RESP_DATA: 0x0000000000000003
# Time: 406568494 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406581.0 ns ACT : Bank = 0 Row = 3
# sdramtoptb.U_SDRAM : at time 406649.0 ns READ : Bank = 0 Row = 3, Col = 44, Data = 4
# sdramtoptb.U_SDRAM : at time 406661.0 ns READ : Bank = 0 Row = 3, Col = 45, Data = 0
# sdramtoptb.U_SDRAM : at time 406674.0 ns READ : Bank = 0 Row = 3, Col = 46, Data = 0
# sdramtoptb.U_SDRAM : at time 406686.0 ns READ : Bank = 0 Row = 3, Col = 47, Data = 0
# ** Note: RESP_DATA: 0x0000000000000004
# Time: 406830990 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# ** Note: RESP_CMD: words64=4 op_id=17
```

Рисунок 24. Фрагмент для теста №2

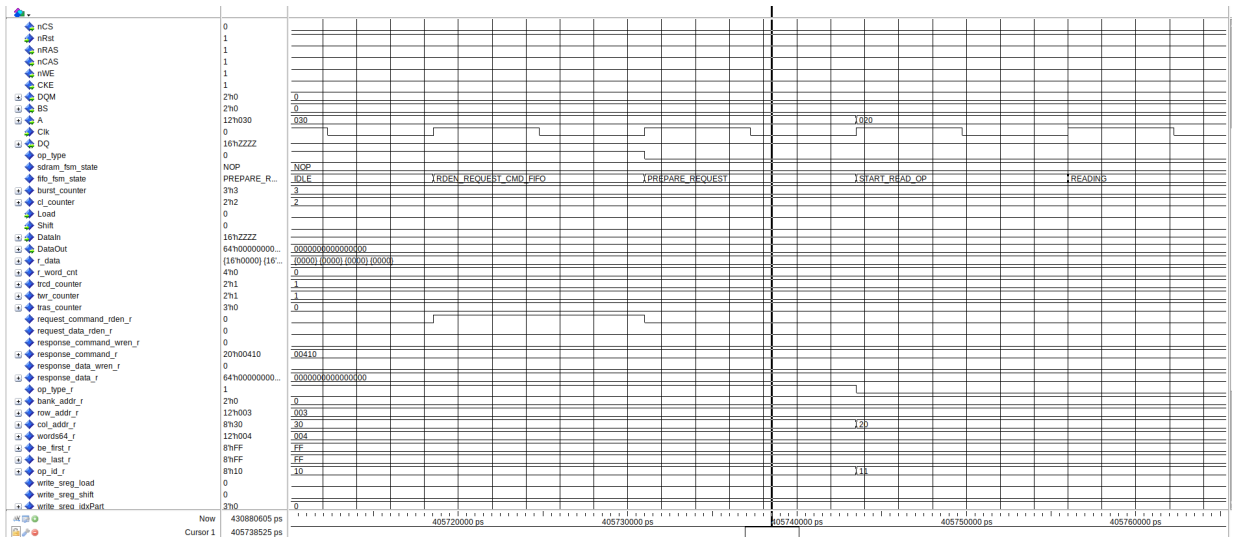


Рисунок 25. Фрагмент временной диаграммы для теста №3

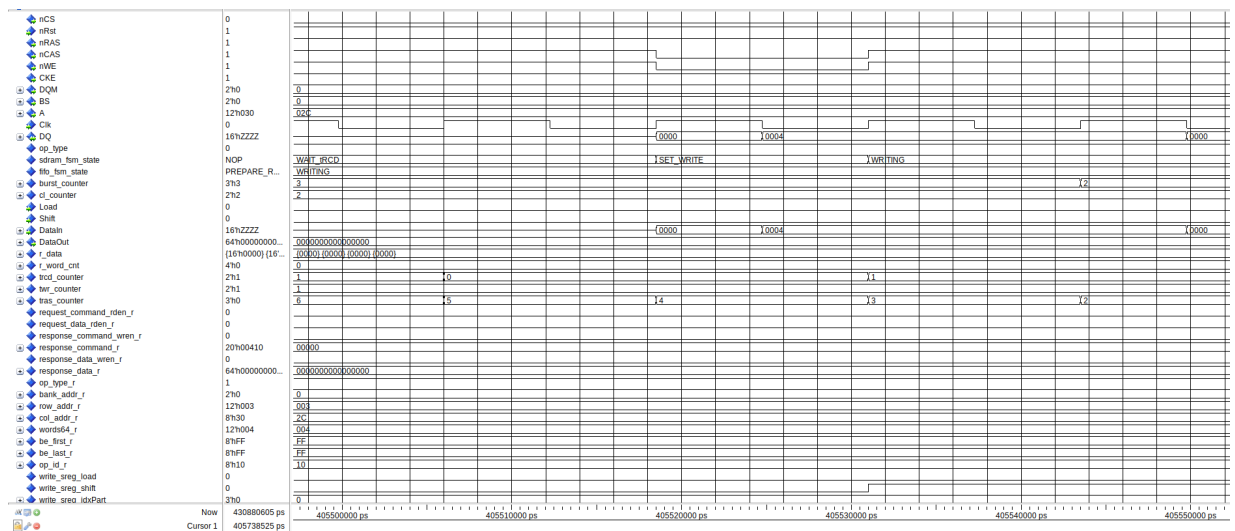


Рисунок 26. Фрагмент временной диаграммы для теста №4

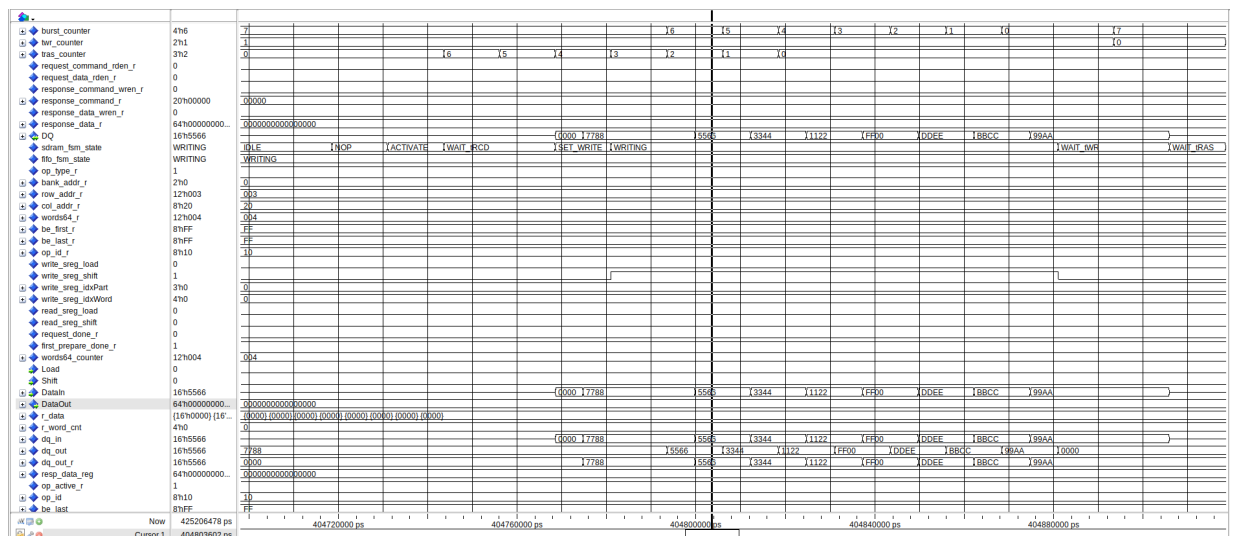


Рисунок 27. Фрагмент временной диаграммы для теста №5

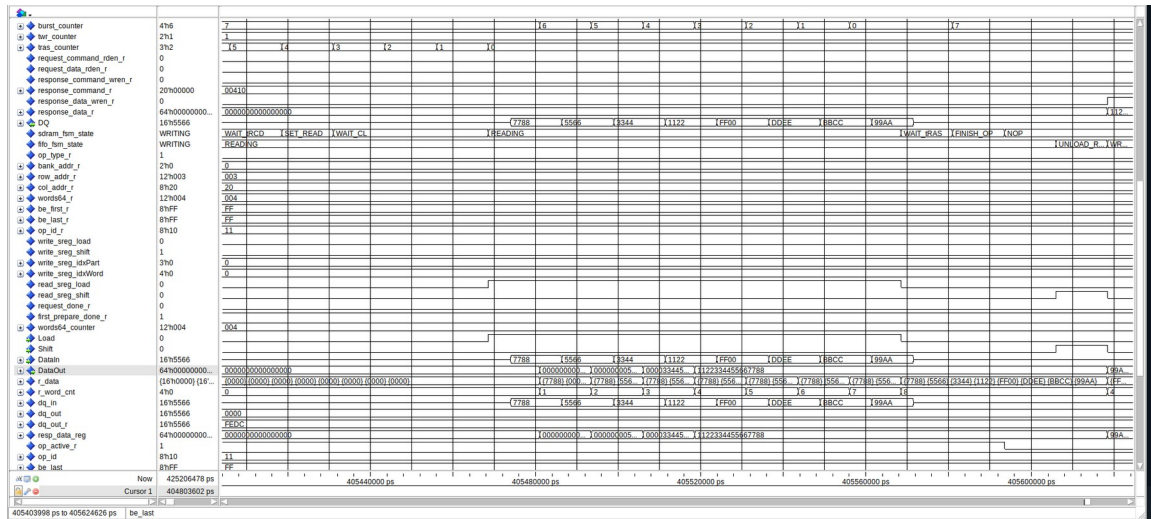


Рисунок 28. Фрагмент временной диаграммы для теста №6

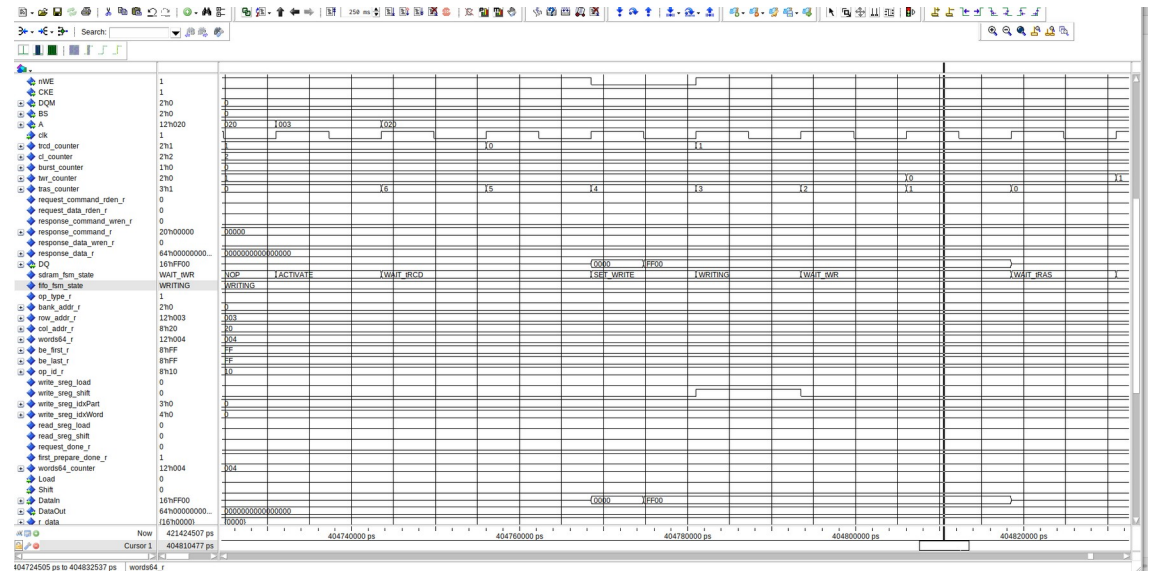


Рисунок 29. Фрагмент временной диаграммы для теста №7


```

# sdramtoptb.U_SDRAM : at time 404106.0 ns AREF : Auto Refresh
# sdramtoptb.U_SDRAM : at time 404231.0 ns AREF : Auto Refresh
# sdramtoptb.U_SDRAM : at time 404356.0 ns AREF : Auto Refresh
# sdramtoptb.U_SDRAM : at time 404481.0 ns AREF : Auto Refresh
# sdramtoptb.U_SDRAM : at time 404606.0 ns AREF : Auto Refresh
# sdramtoptb.U_SDRAM : at time 404756.0 ns ACT : Bank = 3 Row = 4095
# sdramtoptb.U_SDRAM : at time 404794.0 ns WRITE: Bank = 3 Row = 4095, Col = 252, Data = 30600
# sdramtoptb.U_SDRAM : at time 404806.0 ns WRITE: Bank = 3 Row = 4095, Col = 253, Data = 21862
# sdramtoptb.U_SDRAM : at time 404819.0 ns WRITE: Bank = 3 Row = 4095, Col = 254, Data = 13124
# sdramtoptb.U_SDRAM : at time 404831.0 ns WRITE: Bank = 3 Row = 4095, Col = 255, Data = 4386
# sdramtoptb.U_SDRAM : at time 405006.0 ns ACT : Bank = 0 Row = 0
# sdramtoptb.U_SDRAM : at time 405044.0 ns WRITE: Bank = 0 Row = 0, Col = 0, Data = 65280
# sdramtoptb.U_SDRAM : at time 405056.0 ns WRITE: Bank = 0 Row = 0, Col = 1, Data = 56814
# sdramtoptb.U_SDRAM : at time 405069.0 ns WRITE: Bank = 0 Row = 0, Col = 2, Data = 48076
# sdramtoptb.U_SDRAM : at time 405081.0 ns WRITE: Bank = 0 Row = 0, Col = 3, Data = 39338
# sdramtoptb.U_SDRAM : at time 405256.0 ns ACT : Bank = 0 Row = 0
# sdramtoptb.U_SDRAM : at time 405294.0 ns WRITE: Bank = 0 Row = 0, Col = 4, Data = 52719
# sdramtoptb.U_SDRAM : at time 405306.0 ns WRITE: Bank = 0 Row = 0, Col = 5, Data = 35243
# sdramtoptb.U_SDRAM : at time 405319.0 ns WRITE: Bank = 0 Row = 0, Col = 6, Data = 17767
# sdramtoptb.U_SDRAM : at time 405331.0 ns WRITE: Bank = 0 Row = 0, Col = 7, Data = 291
# sdramtoptb.U_SDRAM : at time 405506.0 ns ACT : Bank = 0 Row = 0
# sdramtoptb.U_SDRAM : at time 405544.0 ns WRITE: Bank = 0 Row = 0, Col = 8, Data = 12816
# sdramtoptb.U_SDRAM : at time 405556.0 ns WRITE: Bank = 0 Row = 0, Col = 9, Data = 30292
# sdramtoptb.U_SDRAM : at time 405569.0 ns WRITE: Bank = 0 Row = 0, Col = 10, Data = 47768
# sdramtoptb.U_SDRAM : at time 405581.0 ns WRITE: Bank = 0 Row = 0, Col = 11, Data = 65244
# ** Note: RESP_CMD: words64=4 op_id=16
# Time: 405781007 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 405794.0 ns ACT : Bank = 3 Row = 4095
# sdramtoptb.U_SDRAM : at time 405861.0 ns READ : Bank = 3 Row = 4095, Col = 252, Data = 30600
# sdramtoptb.U_SDRAM : at time 405874.0 ns READ : Bank = 3 Row = 4095, Col = 253, Data = 21862
# sdramtoptb.U_SDRAM : at time 405886.0 ns READ : Bank = 3 Row = 4095, Col = 254, Data = 13124
# sdramtoptb.U_SDRAM : at time 405899.0 ns READ : Bank = 3 Row = 4095, Col = 255, Data = 4386
# ** Note: RESP_DATA: 0x1122334455667788
# Time: 406043503 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406056.0 ns ACT : Bank = 0 Row = 0
# sdramtoptb.U_SDRAM : at time 406124.0 ns READ : Bank = 0 Row = 0, Col = 0, Data = 65280
# sdramtoptb.U_SDRAM : at time 406136.0 ns READ : Bank = 0 Row = 0, Col = 1, Data = 56814
# sdramtoptb.U_SDRAM : at time 406149.0 ns READ : Bank = 0 Row = 0, Col = 2, Data = 48076
# sdramtoptb.U_SDRAM : at time 406161.0 ns READ : Bank = 0 Row = 0, Col = 3, Data = 39338
# ** Note: RESP_DATA: 0x99AABBCCDDEEFF00
# Time: 406305999 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406318.0 ns ACT : Bank = 0 Row = 0
# sdramtoptb.U_SDRAM : at time 406386.0 ns READ : Bank = 0 Row = 0, Col = 4, Data = 52719
# sdramtoptb.U_SDRAM : at time 406399.0 ns READ : Bank = 0 Row = 0, Col = 5, Data = 35243
# sdramtoptb.U_SDRAM : at time 406411.0 ns READ : Bank = 0 Row = 0, Col = 6, Data = 17767
# sdramtoptb.U_SDRAM : at time 406424.0 ns READ : Bank = 0 Row = 0, Col = 7, Data = 291
# ** Note: RESP_DATA: 0x0123456789ABCDEF
# Time: 406568494 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406581.0 ns ACT : Bank = 0 Row = 0
# sdramtoptb.U_SDRAM : at time 406649.0 ns READ : Bank = 0 Row = 0, Col = 8, Data = 12816
# sdramtoptb.U_SDRAM : at time 406661.0 ns READ : Bank = 0 Row = 0, Col = 9, Data = 30292
# sdramtoptb.U_SDRAM : at time 406674.0 ns READ : Bank = 0 Row = 0, Col = 10, Data = 47768
# sdramtoptb.U_SDRAM : at time 406686.0 ns READ : Bank = 0 Row = 0, Col = 11, Data = 65244
# ** Note: RESP_DATA: 0xFEDCBA9876543210
# Time: 406830990 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# ** Note: RESP_CMD: words64=4 op_id=17
# Time: 406893489 ps Iteration: 9 Instance: /sdramtoptb/U_TESTER
# sdramtoptb.U_SDRAM : at time 406906.0 ns AREF : Auto Refresh

```

Рисунок 31. Фрагмент для теста №9

Синтез и компиляция схемы

После успешного прохождения тестирования разрабатываемой подсистемы в программе Quartus были проведены компиляция, отчёт о которой представлен на Рисунке 33, и синтез RTL-схемы, показанной на Рисунке 34.

Flow Status	Successful - Tue Feb 17 15:49:02 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	sdram_fsm
Top-level Entity Name	SdramTopWrapper
Family	Cyclone 10 LP
Device	10CL025YU256C8G
Timing Models	Final
Total logic elements	790 / 24,624 (3 %)
Total registers	630
Total pins	50 / 151 (33 %)
Total virtual pins	0
Total memory bits	50,688 / 608,256 (8 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	1 / 4 (25 %)

Рисунок 33. Отчёт о компиляции

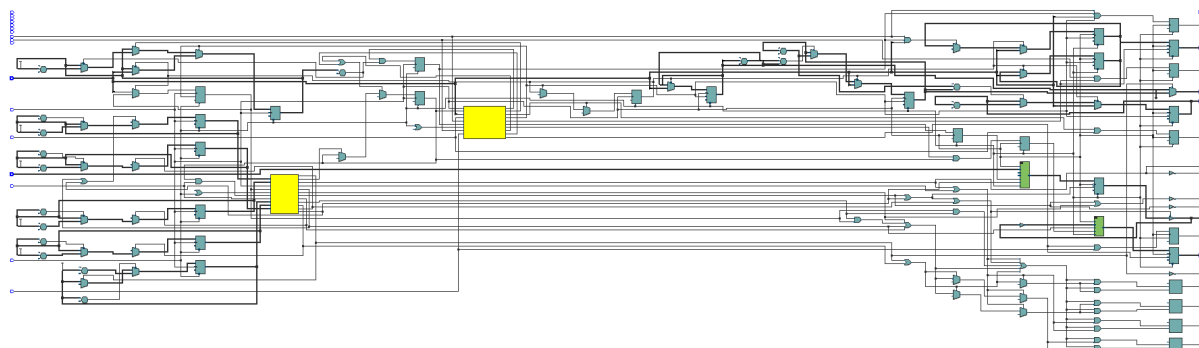


Рисунок 34. RTL-схема, полученная в ходе синтеза

Результаты прошивки ПЛИС

Соответствие выводов ПЛИС и логических сигналов схемы представлена на Рисунке 35.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation
A[11]	Output	PIN_E8	8	B8_N0	PIN_E8	2.5 V		8mA (default)	2 (default)		
A[10]	Output	PIN_A5	8	B8_N0	PIN_A5	2.5 V		8mA (default)	2 (default)		
A[9]	Output	PIN_D8	8	B8_N0	PIN_D8	2.5 V		8mA (default)	2 (default)		
A[8]	Output	PIN_D6	8	B8_N0	PIN_D6	2.5 V		8mA (default)	2 (default)		
A[7]	Output	PIN_E7	8	B8_N0	PIN_E7	2.5 V		8mA (default)	2 (default)		
A[6]	Output	PIN_E6	8	B8_N0	PIN_E6	2.5 V		8mA (default)	2 (default)		
A[5]	Output	PIN_D3	8	B8_N0	PIN_D3	2.5 V		8mA (default)	2 (default)		
A[4]	Output	PIN_C3	8	B8_N0	PIN_C3	2.5 V		8mA (default)	2 (default)		
A[3]	Output	PIN_B3	8	B8_N0	PIN_B3	2.5 V		8mA (default)	2 (default)		
A[2]	Output	PIN_B4	8	B8_N0	PIN_B4	2.5 V		8mA (default)	2 (default)		
A[1]	Output	PIN_B5	8	B8_N0	PIN_B5	2.5 V		8mA (default)	2 (default)		
A[0]	Output	PIN_A3	8	B8_N0	PIN_A3	2.5 V		8mA (default)	2 (default)		
BS[1]	Output	PIN_B6	8	B8_N0	PIN_B6	2.5 V		8mA (default)	2 (default)		
BS[0]	Output	PIN_A4	8	B8_N0	PIN_A4	2.5 V		8mA (default)	2 (default)		
CKE	Output	PIN_F8	8	B8_N0	PIN_F8	2.5 V		8mA (default)	2 (default)		
CLK_12MHz	Input	PIN_M2	2	B2_N0	PIN_M2	2.5 V		8mA (default)			
CLK_160MHz_DBG	Output	PIN_R13	4	B4_N0	PIN_R13	2.5 V		8mA (default)	2 (default)		
Dq[15]	Bidir	PIN_A14	7	B7_N0	PIN_A14	2.5 V		8mA (default)	2 (default)		
Dq[14]	Bidir	PIN_C14	7	B7_N0	PIN_C14	2.5 V		8mA (default)	2 (default)		
Dq[13]	Bidir	PIN_F9	7	B7_N0	PIN_F9	2.5 V		8mA (default)	2 (default)		
Dq[12]	Bidir	PIN_D14	7	B7_N0	PIN_D14	2.5 V		8mA (default)	2 (default)		
Dq[11]	Bidir	PIN_E9	7	B7_N0	PIN_E9	2.5 V		8mA (default)	2 (default)		
Dq[10]	Bidir	PIN_A15	7	B7_N0	PIN_A15	2.5 V		8mA (default)	2 (default)		
Dq[9]	Bidir	PIN_E11	7	B7_N0	PIN_E11	2.5 V		8mA (default)	2 (default)		
Dq[8]	Bidir	PIN_D11	7	B7_N0	PIN_D11	2.5 V		8mA (default)	2 (default)		
Dq[7]	Bidir	PIN_C9	7	B7_N0	PIN_C9	2.5 V		8mA (default)	2 (default)		
Dq[6]	Bidir	PIN_B12	7	B7_N0	PIN_B12	2.5 V		8mA (default)	2 (default)		
Dq[5]	Bidir	PIN_D9	7	B7_N0	PIN_D9	2.5 V		8mA (default)	2 (default)		
Dq[4]	Bidir	PIN_A12	7	B7_N0	PIN_A12	2.5 V		8mA (default)	2 (default)		
Dq[3]	Bidir	PIN_A11	7	B7_N0	PIN_A11	2.5 V		8mA (default)	2 (default)		
Dq[2]	Bidir	PIN_B11	7	B7_N0	PIN_B11	2.5 V		8mA (default)	2 (default)		
Dq[1]	Bidir	PIN_A10	7	B7_N0	PIN_A10	2.5 V		8mA (default)	2 (default)		
Dq[0]	Bidir	PIN_B10	7	B7_N0	PIN_B10	2.5 V		8mA (default)	2 (default)		
DO0_DBG	Output	PIN_C15	6	B6_N0	PIN_C15	2.5 V		8mA (default)	2 (default)		
DOM[1]	Output	PIN_D12	7	B7_N0	PIN_D12	2.5 V		8mA (default)	2 (default)		
DOM[0]	Output	PIN_B13	7	B7_N0	PIN_B13	2.5 V		8mA (default)	2 (default)		
FSM_ACTIVE_DBG	Output	PIN_B16	6	B6_N0	PIN_B16	2.5 V		8mA (default)	2 (default)		
nCAS	Output	PIN_C8	8	B8_N0	PIN_C8	2.5 V		8mA (default)	2 (default)		
nCAS_DBG	Output	PIN_F13	6	B6_N0	PIN_F13	2.5 V		8mA (default)	2 (default)		
nCS	Output	PIN_A6	8	B8_N0	PIN_A6	2.5 V		8mA (default)	2 (default)		
nCS_DBG	Output	PIN_F15	6	B6_N0	PIN_F15	2.5 V		8mA (default)	2 (default)		
nRAS	Output	PIN_B7	8	B8_N0	PIN_B7	2.5 V		8mA (default)	2 (default)		
nRAS_DBG	Output	PIN_D15	6	B6_N0	PIN_D15	2.5 V		8mA (default)	2 (default)		
nRst	Input	PIN_N6	3	B3_N0	PIN_N6	2.5 V		8mA (default)			
nWE	Output	PIN_A7	8	B8_N0	PIN_A7	2.5 V		8mA (default)	2 (default)		
nWE_DBG	Output	PIN_D16	6	B6_N0	PIN_D16	2.5 V		8mA (default)	2 (default)		
RD_LOAD_DBG	Output	PIN_T13	4	B4_N0	PIN_T13	2.5 V		8mA (default)	2 (default)		
RESP_VALID_DBG	Output	PIN_F16	6	B6_N0	PIN_F16	2.5 V		8mA (default)	2 (default)		
WR_SHIFT_DBG	Output	PIN_R12	4	B4_N0	PIN_R12	2.5 V		8mA (default)	2 (default)		
WREN	Output	PIN_T14	4	B4_N0	PIN_T14	2.5 V		8mA (default)	2 (default)		

Рисунок 35. Соответствие выводов ПЛИС и логических сигналов схемы

Была выполнена прошивка ПЛИС и исследование работы подсистемы на реальной плате на осциллографе. Результаты представлены на Рисунках 36-39.

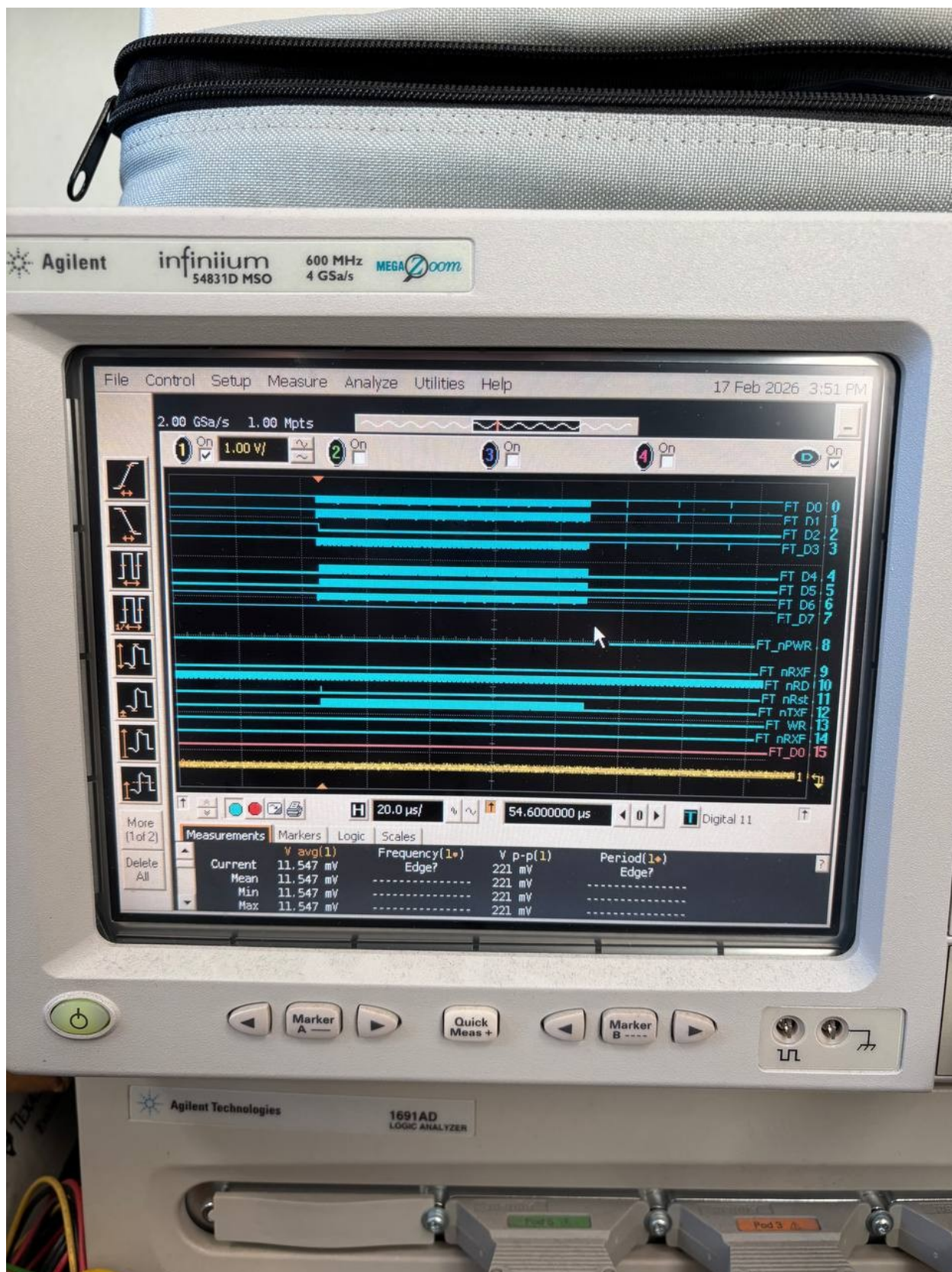


Рисунок 36. Общий вид работы контроллера

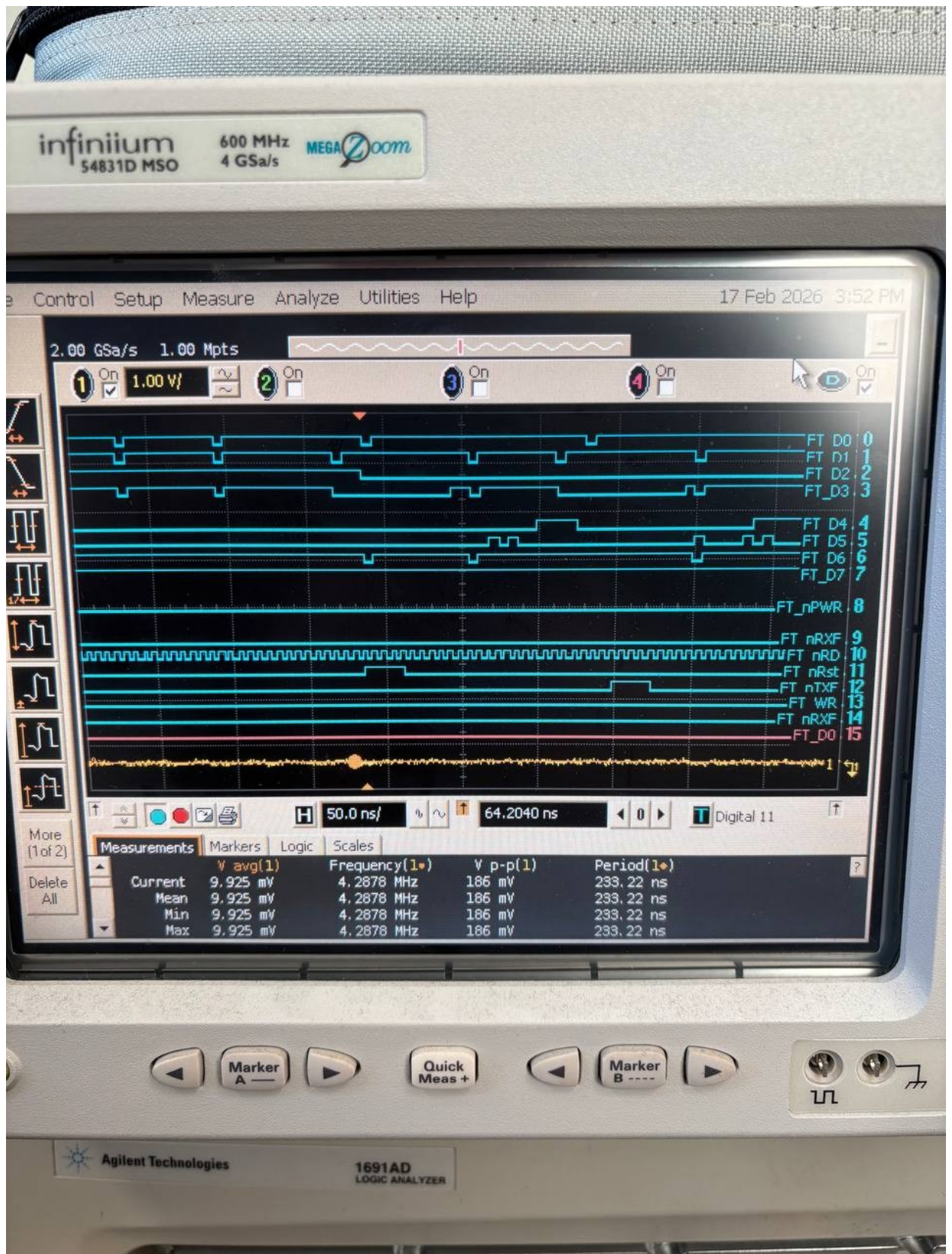


Рисунок 37. Приблизённый вид операций записи в память и чтения из неё

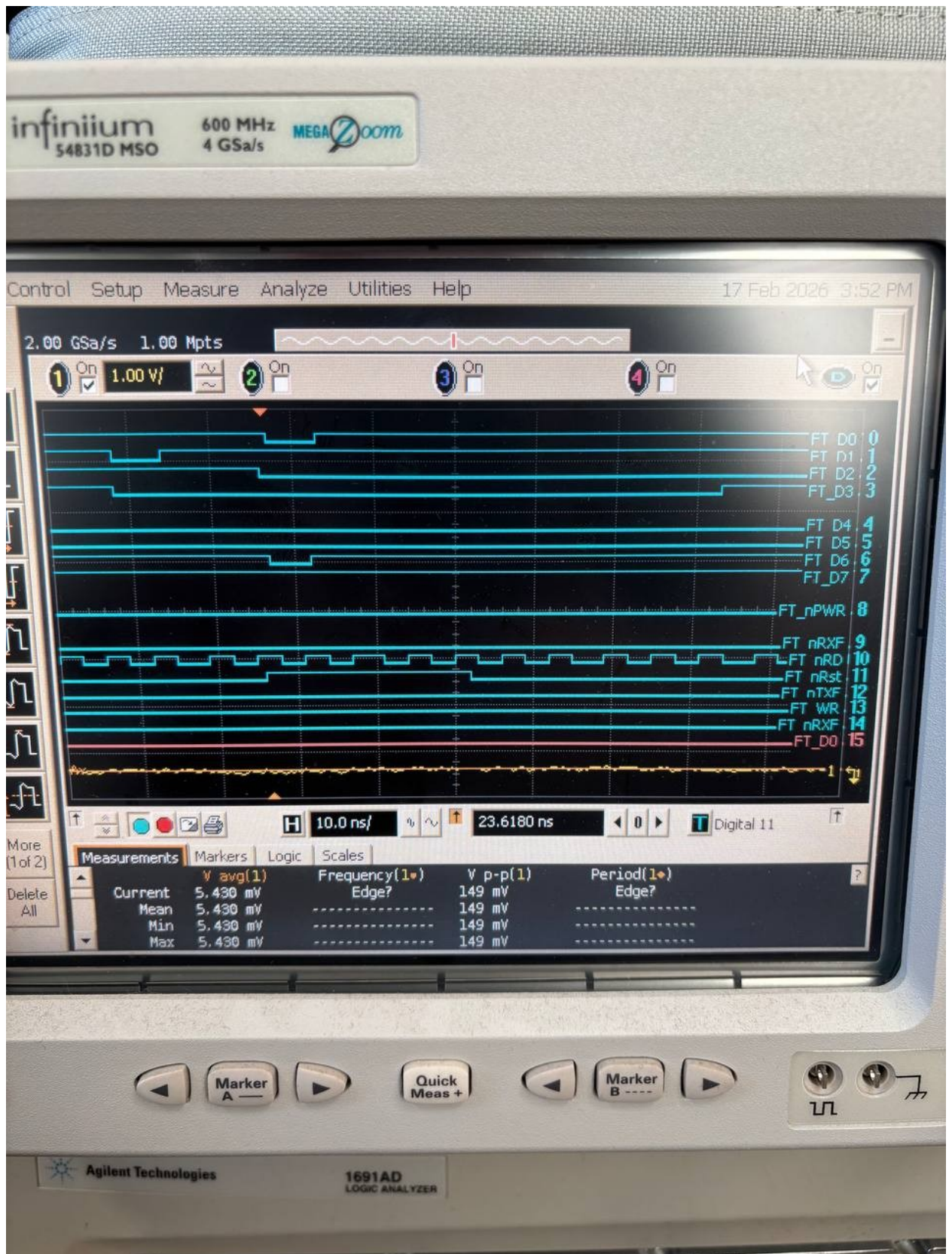


Рисунок 38. Операция записи, происходящая за 4 такта

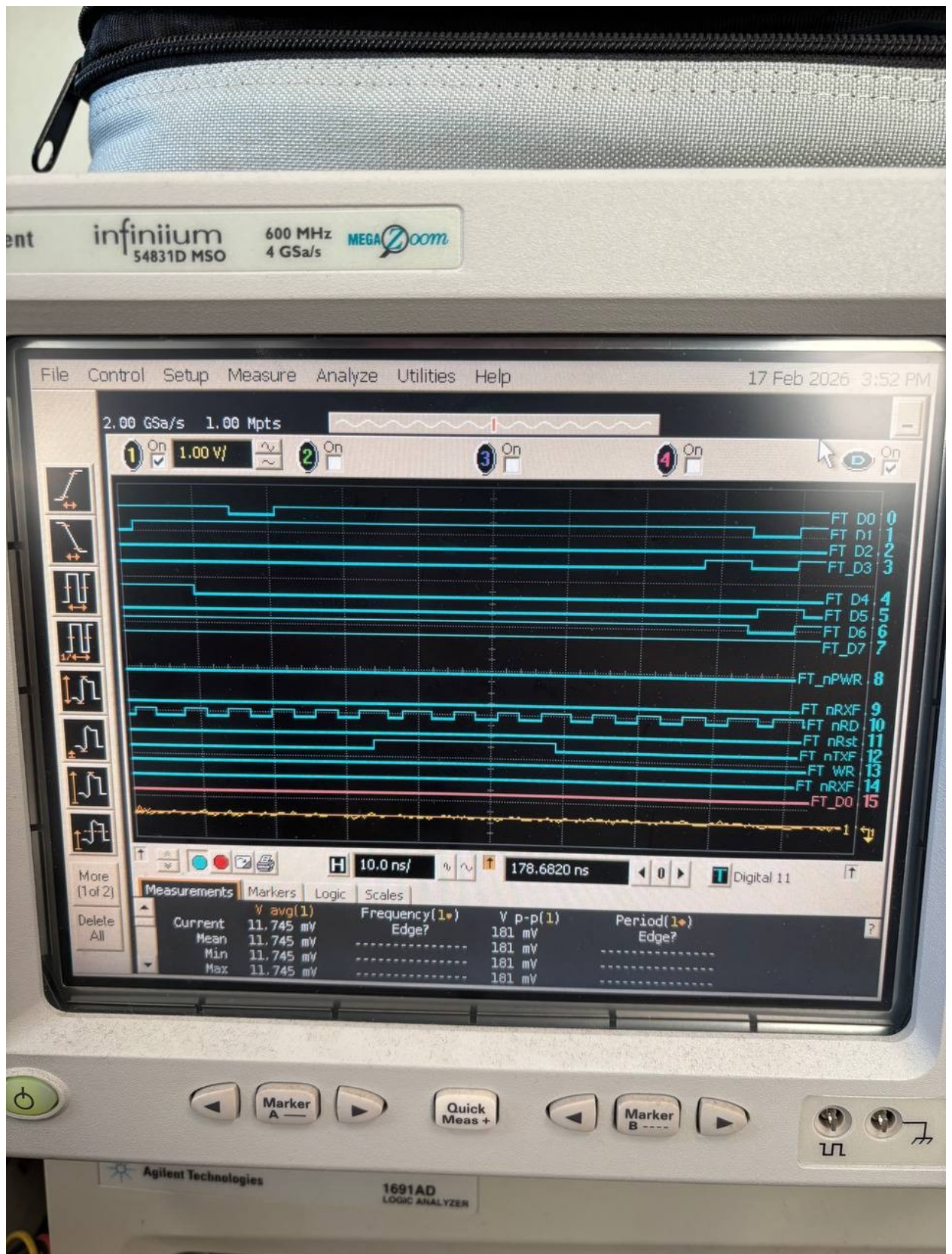


Рисунок 39. Операция чтения, происходящая за 4 такта

Заключение

В ходе данной курсовой работы был разработан конечный автомат для обмена с внешней памятью SDRAM и выполнения присвоений выходных сигналов в зависимости от текущего состояния конечного автомата. Были описаны все его состояния, разработаны дополнительные модули и проведены тестирование и синтез.

Приложение

Приложение 1. Файл write_shift_reg.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity write_shift_reg is
    generic (
        WORD_WIDTH : integer := 16; -- 8, 16, 32, 64
        BURST       : integer := 4  -- 1, 2, 4, 8
    );
    port (
        Clk      : in  std_logic;
        nRst     : in  std_logic;

        Load     : in  std_logic;
        Shift    : in  std_logic;

        Idx       : in  std_logic_vector(2 downto 0); -- для длины регистра < 64, вычисляется
        64/FRAG_BITS, 000 -> 001 -> 002 ...
        ByteEn    : in  std_logic_vector(7 downto 0);
        DataIn    : in  std_logic_vector(63 downto 0);

        Enable    : out std_logic_vector(WORD_WIDTH/8-1 downto 0);
        WordOut   : out std_logic_vector(WORD_WIDTH-1 downto 0)
    );
end entity;

architecture rtl of write_shift_reg is

    type t_sreg_data   is array (0 to BURST-1) of std_logic_vector(WORD_WIDTH-1 downto 0);
    type t_sreg_enable is array (0 to BURST-1) of std_logic_vector(WORD_WIDTH/8-1 downto 0);

    signal r_data      : t_sreg_data;
    signal r_enable    : t_sreg_enable;
```

```

constant FRAG_BITS    : integer := WORD_WIDTH * BURST;

constant BYTES_PER_W  : integer := WORD_WIDTH / 8;    -- 1,2,4,8

constant WORDS_PER_64 : integer := 64 / WORD_WIDTH;  -- 1,2,4,8

begin

    WordOut <= r_data(0);
    Enable  <= r_enable(0);

    p_sreg : process(Clk, nRst)
    begin
        if nRst = '0' then
            for j in 0 to BURST-1 loop
                r_data(j)    <= (others => '0');
                r_enable(j) <= (others => '0');
            end loop;

            elsif rising_edge(Clk) then

                if Shift = '1' then
                    for j in 0 to BURST-2 loop
                        r_data(j)    <= r_data(j+1);
                        r_enable(j) <= r_enable(j+1);
                    end loop;
                    r_data(BURST-1)  <= (others => '0');
                    r_enable(BURST-1) <= (others => '0');

                elsif Load = '1' then

                    if FRAG_BITS <= 64 then
                        for j in 0 to BURST-1 loop
                            if ((conv_integer(Idx)*BURST + j + 1) * WORD_WIDTH) <= 64 then
                                r_data(j) <= DataIn(
                                    ((conv_integer(Idx)*BURST + j + 1) * WORD_WIDTH - 1) downto
                                    ((conv_integer(Idx)*BURST + j)      * WORD_WIDTH));

                                r_enable(j) <= ByteEn(
                                    ((conv_integer(Idx)*BURST + j + 1) * BYTES_PER_W - 1) downto

```

```

((conv_integer(Idc)*BURST + j) * BYTES_PER_W));

else
    r_data(j) <= (others => '0');
    r_enable(j) <= (others => '0');
end if;
end loop;

else
    for j in 0 to BURST-1 loop
        if j < WORDS_PER_64 then
            r_data(j) <= DataIn(((j+1) * WORD_WIDTH - 1) downto (j * WORD_WIDTH));
            r_enable(j) <= ByteEn(((j+1) * BYTES_PER_W - 1) downto (j *
BYTES_PER_W));
        else
            r_data(j) <= (others => '0');
            r_enable(j) <= (others => '0');
        end if;
    end loop;
end if;
end if;
end process;

end architecture;

```

Приложение 2. Файл write_shift_reg_tb.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity write_shift_reg_tb is
end entity;

architecture top of write_shift_reg_tb is
begin

    T_8_1 : entity work.write_shift_reg_tester
        generic map ( WORD_WIDTH => 8, BURST => 1 );

```



```

T_8_2 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 2 );

T_8_4 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 4 );

T_8_8 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 8 );

T_16_1 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 1 );

T_16_2 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 2 );

T_16_4 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 4 );

T_16_8 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 8 );

T_32_1 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 1 );

T_32_2 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 2 );

T_32_4 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 4 );

T_32_8 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 8 );

T_64_1 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 64, BURST => 1 );

T_64_2 : entity work.write_shift_reg_tester

```

```

        generic map ( WORD_WIDTH => 64, BURST => 2 );

T_64_4 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 64, BURST => 4 );

T_64_8 : entity work.write_shift_reg_tester
    generic map ( WORD_WIDTH => 64, BURST => 8 );

end architecture;

```

Приложение 3. Файл write_shift_reg_tester.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity write_shift_reg_tester is
    generic (
        WORD_WIDTH : integer := 16;
        BURST      : integer := 4
    );
end entity;

architecture tb of write_shift_reg_tester is

    constant CLK_PERIOD    : time      := 10 ns;
    constant FRAG_BITS     : integer := WORD_WIDTH * BURST;
    constant WORDS_PER_64 : integer := 64 / WORD_WIDTH;

    function calc_num_frags return integer is
    begin
        if FRAG_BITS = 0 then
            return 1;
        elsif FRAG_BITS >= 64 then
            return 1;
        else
            return 64 / FRAG_BITS;
        end if;
    end function;
end architecture;

```

```

    end if;
end function;

constant NUM_FRAGS : integer := calc_num_frgs;

type t_idx_array is array (0 to 7) of std_logic_vector(2 downto 0);
constant IDX_TABLE : t_idx_array := (
    "000", "001", "010", "011",
    "100", "101", "110", "111"
);

constant DATA_TEMPLATE : std_logic_vector(63 downto 0) :=
    x"0123456789ABCDEF";

type t_be_array is array (natural range <>) of std_logic_vector(7 downto 0);

constant BE_PATTERNS : t_be_array := (
    "11111111",
    "00111111",
    "11111000",
    "01111100"
);

signal Clk      : std_logic := '0';
signal nRst     : std_logic := '0';

signal Load     : std_logic := '0';
signal Shift    : std_logic := '0';

signal Idx      : std_logic_vector(2 downto 0) := (others => '0');
signal ByteEn   : std_logic_vector(7 downto 0) := (others => '0');
signal DataIn   : std_logic_vector(63 downto 0) := (others => '0');

signal Enable   : std_logic_vector(WORD_WIDTH/8-1 downto 0);
signal WordOut  : std_logic_vector(WORD_WIDTH-1 downto 0);

begin

```

```

p_clk : process
begin
    Clk <= '0';
    wait for CLK_PERIOD/2;
    Clk <= '1';
    wait for CLK_PERIOD/2;
end process;

dut : entity work.write_shift_reg
    generic map (
        WORD_WIDTH => WORD_WIDTH,
        BURST      => BURST
    )
    port map (
        Clk      => Clk,
        nRst     => nRst,
        Load     => Load,
        Shift    => Shift,
        Idx      => Idx,
        ByteEn   => ByteEn,
        DataIn   => DataIn,
        Enable   => Enable,
        WordOut  => WordOut
    );

p_stim : process

    procedure set_pattern(
        constant be_idx : in integer
    ) is
    begin
        if be_idx < BE_PATTERNS'length then
            ByteEn <= BE_PATTERNS(be_idx);
        else
            ByteEn <= (others => '1');
        end if;
        DataIn <= DATA_TEMPLATE;
    end procedure;

```

```

begin

    nRst    <= '0';
    Load    <= '0';
    Shift    <= '0';
    Idx      <= (others => '0');
    ByteEn   <= (others => '0');
    DataIn   <= DATA_TEMPLATE;

    wait for 3*CLK_PERIOD;
    wait until rising_edge(Clk);
    nRst <= '1';

    wait for 2*CLK_PERIOD;

    for be_i in 0 to BE_PATTERNS'length-1 loop

        set_pattern(be_i);
        wait until rising_edge(Clk);

        if FRAG_BITS >= 64 then

            Idx  <= IDX_TABLE(0);
            Load <= '1';
            Shift <= '0';
            wait until rising_edge(Clk);
            Load <= '0';

            for w in 0 to WORDS_PER_64-1 loop
                Shift <= '1';
                wait until rising_edge(Clk);
            end loop;
            Shift <= '0';

            wait for 4*CLK_PERIOD;

        else

```

```

    Idx  <= IDX_TABLE(0);

    Load <= '1';

    Shift <= '0';

    wait until rising_edge(Clk);

    Load <= '0';

    for frag in 0 to NUM_FRAGS-1 loop

        for k in 0 to BURST-1 loop

            Shift <= '1';

            wait until rising_edge(Clk);

        end loop;

        Shift <= '0';

        if frag < NUM_FRAGS-1 then

            Idx  <= IDX_TABLE(frag + 1);

            Load <= '1';

            wait until rising_edge(Clk);

            Load <= '0';

        end if;

    end loop;

    wait for 6*CLK_PERIOD;

end if;

Load  <= '0';

Shift <= '0';

wait for 5*CLK_PERIOD;

end loop;

wait;

end process;

end architecture;

```

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity read_shift_reg is
    generic (
        WORD_WIDTH : integer := 16; -- 8, 16, 32, 64
        BURST       : integer := 4  -- 1, 2, 4, 8
    );
    port (
        Clk       : in  std_logic;
        nRst      : in  std_logic;

        Load      : in  std_logic;
        Shift      : in  std_logic;

        DataIn     : in  std_logic_vector(WORD_WIDTH-1 downto 0);

        DataOut    : out std_logic_vector(63 downto 0);
        Valid      : out std_logic
    );
end entity;

architecture rtl of read_shift_reg is

    type t_sreg_data is array (0 to BURST-1) of std_logic_vector(WORD_WIDTH-1 downto 0);
    signal r_data : t_sreg_data;
    signal r_word_cnt : std_logic_vector(3 downto 0);

    constant FRAG_BITS    : integer := WORD_WIDTH * BURST;
    constant WORDS_PER_64 : integer := 64 / WORD_WIDTH;

begin

    gen_data_out_small : if BURST <= WORDS_PER_64 generate

        gen_real : for i in 0 to BURST-1 generate

```

```

        DataOut((i+1)*WORD_WIDTH - 1 downto i*WORD_WIDTH) <= r_data(i);
    end generate;

    gen_zero : for i in BURST to WORDS_PER_64-1 generate
        DataOut((i+1)*WORD_WIDTH - 1 downto i*WORD_WIDTH) <= (others => '0');
    end generate;

end generate;

gen_data_out_large : if BURST > WORDS_PER_64 generate

    gen_real_only : for i in 0 to WORDS_PER_64-1 generate
        DataOut((i+1)*WORD_WIDTH - 1 downto i*WORD_WIDTH) <= r_data(i);
    end generate;

end generate;

pad_hi : if WORDS_PER_64*WORD_WIDTH < 64 generate
    DataOut(63 downto WORDS_PER_64*WORD_WIDTH) <= (others => '0');
end generate;

Valid <=
    '1' when (
        (FRAG_BITS >= 64 and conv_integer(r_word_cnt) >= WORDS_PER_64) or
        (FRAG_BITS < 64 and conv_integer(r_word_cnt) = BURST)
    ) else '0';

p_sreg : process(Clk, nRst)
begin
    if nRst = '0' then
        for j in 0 to BURST-1 loop
            r_data(j) <= (others => '0');
        end loop;
        r_word_cnt <= (others => '0');

    elsif rising_edge(Clk) then

        if (Shift = '1') and

```



```

((FRAG_BITS >= 64 and conv_integer(r_word_cnt) >= WORDS_PER_64) or
 (FRAG_BITS < 64 and conv_integer(r_word_cnt) = BURST)) then

if FRAG_BITS >= 64 then
  for j in 0 to BURST-1 loop
    if j <= BURST-WORDS_PER_64-1 then
      r_data(j) <= r_data(j+WORDS_PER_64);
    else
      r_data(j) <= (others => '0');
    end if;
  end loop;
  r_word_cnt <= r_word_cnt - conv_std_logic_vector(WORDS_PER_64, 4);

else
  for j in 0 to BURST-1 loop
    r_data(j) <= (others => '0');
  end loop;
  r_word_cnt <= r_word_cnt - conv_std_logic_vector(BURST, 4);
end if;

elsif (Load = '1') and (conv_integer(r_word_cnt) < BURST) then
  r_data(conv_integer(r_word_cnt)) <= DataIn;
  r_word_cnt <= r_word_cnt + conv_std_logic_vector(1, 4);
end if;
end if;
end process;

end architecture;

```

Приложение 5. Файл read_shift_reg_tb.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity read_shift_reg_tb is
end entity;

architecture sim of read_shift_reg_tb is
begin

```

```

T_8_1 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 1 );

T_8_2 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 2 );

T_8_4 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 4 );

T_8_8 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 8, BURST => 8 );

T_16_1 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 1 );

T_16_2 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 2 );

T_16_4 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 4 );

T_16_8 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 16, BURST => 8 );

T_32_1 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 1 );

T_32_2 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 2 );

T_32_4 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 4 );

T_32_8 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 32, BURST => 8 );

T_64_1 : entity work.read_shift_reg_tester

```

```

        generic map ( WORD_WIDTH => 64, BURST => 1 );

T_64_2 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 64, BURST => 2 );

T_64_4 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 64, BURST => 4 );

T_64_8 : entity work.read_shift_reg_tester
    generic map ( WORD_WIDTH => 64, BURST => 8 );

end architecture;

```

Приложение 6. Файл read_shift_reg_tester.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity read_shift_reg_tester is
    generic (
        WORD_WIDTH : integer := 16; -- 8, 16, 32, 64
        BURST      : integer := 4  -- 1, 2, 4, 8
    );
end entity;

architecture tb of read_shift_reg_tester is

    constant CLK_PERIOD : time    := 10 ns;
    constant FRAG_BITS  : integer := WORD_WIDTH * BURST;
    constant NUM_SHIFTS : integer := (FRAG_BITS + 63) / 64;
    constant WORDS_PER_64: integer := 64 / WORD_WIDTH;

    signal Clk      : std_logic := '0';
    signal nRst     : std_logic := '0';

    signal Load     : std_logic := '0';
    signal Shift     : std_logic := '0';

```

```

signal DataIn    : std_logic_vector(WORD_WIDTH-1 downto 0) := (others => '0');
signal DataOut   : std_logic_vector(63 downto 0);
signal Valid     : std_logic;

begin

U_DUT : entity work.read_shift_reg
  generic map (
    WORD_WIDTH => WORD_WIDTH,
    BURST      => BURST
  )
  port map (
    Clk      => Clk,
    nRst     => nRst,
    Load    => Load,
    Shift    => Shift,
    DataIn   => DataIn,
    DataOut  => DataOut,
    Valid    => Valid
  );

p_clk : process
begin
  Clk <= '0';
  wait for CLK_PERIOD/2;
  Clk <= '1';
  wait for CLK_PERIOD/2;
end process;

p_stim : process
begin
  nRst  <= '0';
  Load  <= '0';
  Shift <= '0';
  DataIn <= (others => '0');

  wait for 3*CLK_PERIOD;

```

```

nRst <= '1';

wait until rising_edge(Clk);

for i in 0 to BURST-1 loop
    Load  <= '1';
    Shift <= '0';
    DataIn <= conv_std_logic_vector(i, WORD_WIDTH);
    wait until rising_edge(Clk);
end loop;

Load <= '0';

for k in 0 to NUM_SHIFTS-1 loop
    Shift <= '1';
    Load  <= '0';
    wait until rising_edge(Clk);
end loop;

Shift <= '0';

wait for 5*CLK_PERIOD;
wait;
end process;

end architecture;

```